GTE Function Appendix

A readable, function-by-function guide to the Rust application behind Guess the Elo.

111 functions · 47 focused Rust modules · one navigable PDF

How to use this appendix

Each function begins on a fresh page and has a PDF bookmark. Open the document outline in your PDF reader to jump directly to a function. Each entry includes its source location, responsibility, signature, implementation excerpt, and a note about how it fits into the larger codebase.

Coverage

main.rs selects an entry point. The api/ modules handle Chess.com archives; chess/ owns PGN, positions, and scoring; cli/ separates terminal input, screens, progress, and exercise checks; lessons/ defines the 24-exercise catalog; and web/ handles HTTP, sessions, game pages, and the book reader. HTML templates and stylesheets live under assets/.

The code examples are taken from the current application source. The companion browser book provides the same function pages with chapter navigation and the surrounding project walkthrough.

Function page 001 of 111: `main::main`

Source: `src/main.rs:7`

Responsibility

Selects the terminal curriculum or browser app from the process arguments.

Signature and data contract

```
fn main()
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn main() {
    let args: Vec<String> = std::env::args().collect();
    if args.iter().any(|arg| arg == "--web") {
        web::serve();
    } else {
        cli::run();
    }
}
```

Where it fits

The implementation lives in the focused `main` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 002 of 111: `api::archives::random_public_game`

Source: `src/api/archives.rs:10`

Responsibility

Loads a player's archive index, checks recent monthly archives, filters usable chess games, and selects one candidate.

Signature and data contract

```
pub fn random_public_game(username: &str) -> Result<ChessGame, String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn random_public_game(username: &str) -> Result<ChessGame, String> {
    let username = validate_username(username)?;
    let archives = archive_urls(&username)?;
    let mut candidates = Vec::new();

    for archive in archives.iter().rev().take(4) {
        let response = get_json(archive)?;
        if let Some(games) = response.get("games").and_then(Value::as_array) {
            candidates.extend(games.iter().filter_map(to_chess_game));
        }
        if !candidates.is_empty() {
            break;
        }
    }

    if candidates.is_empty() {
        return Err(format!("No public chess games were found for {username}."));
    }
    let tick = std::time::SystemTime::now()
        .duration_since(std::time::UNIX_EPOCH)
        .map(|duration| duration.as_nanos() as usize)
        .unwrap_or(0);
    let index = tick % candidates.len();
    Ok(candidates.swap_remove(index))
}
```

Where it fits

The implementation lives in the focused `api::archives` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `game_from_archive_link`, `archive_urls`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 003 of 111: `api::archives::game_from_archive_link`

Source: `src/api/archives.rs:36`

Responsibility

Resolves a pasted public game URL by searching the selected player's recent archive records.

Signature and data contract

```
pub fn game_from_archive_link(username: &str, game_url: &str) -> Result<ChessGame, String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn game_from_archive_link(username: &str, game_url: &str) -> Result<ChessGame, String> {
    let username = validate_username(username)?;
    let requested_url = game_url.trim();
    if !requested_url.starts_with("https://www.chess.com/game/")
        && !requested_url.starts_with("https://chess.com/game/")
    {
        return Err("Paste a public Chess.com game URL.".to_owned());
    }

    let archives = archive_urls(&username)?;
    for archive in archives.iter().rev().take(12) {
        let response = get_json(archive)?;
        let Some(games) = response.get("games").and_then(Value::as_array) else {
            continue;
        };
        for game in games {
            let url = game.get("url").and_then(Value::as_str).unwrap_or_default();
            if url == requested_url || same_game_id(url, requested_url) {
                return to_chess_game(game)
                    .ok_or_else(|| "That archive entry does not include a usable PGN.".to_owned());
            }
        }
    }
    Err(format!(
        "That game was not found in {username}'s recent public archives. Try another username or load a random archive
           game."
    ))
}
```

Where it fits

The implementation lives in the focused `api::archives` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `random_public_game`, `archive_urls`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 004 of 111: `api::archives::archive_urls`

Source: `src/api/archives.rs:64`

Responsibility

Requests and extracts the monthly archive URLs returned for a player.

Signature and data contract

```
fn archive_urls(username: &str) -> Result<Vec<String>, String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn archive_urls(username: &str) -> Result<Vec<String>, String> {
    let url = format!("https://api.chess.com/pub/player/{username}/games/archives");
    let response = get_json(&url)?;
    let Some(archives) = response.get("archives").and_then(Value::as_array) else {
        return Err("Chess.com returned no archive list for that username.".to_owned());
    };
    let urls = archives
        .iter()
        .filter_map(Value::as_str)
        .map(str::to_owned)
        .collect::<Vec<_>>();
    if urls.is_empty() {
        return Err("This Chess.com player has no public archived games.".to_owned());
    }
    Ok(urls)
}
```

Where it fits

The implementation lives in the focused `api::archives` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `random_public_game`, `game_from_archive_link`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 005 of 111: `api::client::get_json`

Source: `src/api/client.rs:7`

Responsibility

Performs a public GET request and turns either JSON or transport errors into a Rust Result.

Signature and data contract

```
pub(super) fn get_json(url: &str) -> Result<Value, String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn get_json(url: &str) -> Result<Value, String> {
    let response = ureq::get(url)
        .set("User-Agent", USER_AGENT)
        .call()
        .map_err(|error| format!("Chess.com API request failed: {error}"))?;
    response
        .into_json::<Value>()
        .map_err(|error| format!("Chess.com returned unreadable JSON: {error}"))
}
```

Where it fits

The implementation lives in the focused `api::client` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 006 of 111: `api::mapping::to_chess_game`

Source: `src/api/mapping.rs:8`

Responsibility

Converts one archive JSON value into the app's shared chess model, preserving absent ratings as None.

Signature and data contract

```
pub(super) fn to_chess_game(record: &Value) -> Option<ChessGame>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn to_chess_game(record: &Value) -> Option<ChessGame> {
    if record
        .get("rules")
        .and_then(Value::as_str)
        .is_some_and(|rules| rules != "chess")
    {
        return None;
    }
    let pgn = record.get("pgn")?.as_str()?;
    let white_rating = record
        .get("white")
        .and_then(|value| value.get("rating"))
        .and_then(Value::as_i64)
        .and_then(|rating| i32::try_from(rating).ok());
    let black_rating = record
        .get("black")
        .and_then(|value| value.get("rating"))
        .and_then(Value::as_i64)
        .and_then(|rating| i32::try_from(rating).ok());
    let white_name = record
        .get("white")
        .and_then(|value| value.get("username"))
        .and_then(Value::as_str);
    let black_name = record
        .get("black")
        .and_then(|value| value.get("username"))
        .and_then(Value::as_str);
    let opening = record
        .get("opening")
        .and_then(Value::as_str)
        .map(str::to_owned)
        .or_else(|| {
            record
                .get("eco")
                .and_then(Value::as_str)
                .map(opening_from_eco)
        });
    let url = record.get("url").and_then(Value::as_str).map(str::to_owned);
    let mut game =
        ChessGame::with_metadata(pgn, white_rating, black_rating, opening, None, url, false);
    if let Some(name) = white_name {
        game.white_name = name.to_owned();
    }
    if let Some(name) = black_name {
        game.black_name = name.to_owned();
    }
    if let Some(time_control) = record.get("time_control").and_then(Value::as_str) {
        game.time_control = time_control.to_owned();
    }
    Some(game)
}
```

Where it fits

The implementation lives in the focused `api::mapping` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 007 of 111: `api::urls::validate_username`

Source: `src/api/urls.rs:3`

Responsibility

Normalizes and validates a public Chess.com username before it becomes part of a request URL.

Signature and data contract

```
pub(super) fn validate_username(username: &str) -> Result<String, String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn validate_username(username: &str) -> Result<String, String> {
    let username = username.trim().to_ascii_lowercase();
    if username.is_empty()
        || username.len() > 25
        || !username
            .chars()
            .all(|character| character.is_ascii_alphanumeric() || character == '_')
    {
        return Err(
            "Enter a Chess.com username using letters, numbers, or underscores.".to_owned(),
        );
    }
    Ok(username)
}
```

Where it fits

The implementation lives in the focused `api::urls` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `same_game_id`, `game_id`, `opening_from_eco`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 008 of 111: `api::urls::same_game_id`

Source: `src/api/urls.rs:18`

Responsibility

Compares the stable final game id instead of requiring two URLs to have identical formatting.

Signature and data contract

```
pub(super) fn same_game_id(first: &str, second: &str) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn same_game_id(first: &str, second: &str) -> bool {
    let first_id = game_id(first);
    !first_id.is_empty() && first_id == game_id(second)
}
```

Where it fits

The implementation lives in the focused `api::urls` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `validate_username`, `game_id`, `opening_from_eco`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 009 of 111: `api::urls::game_id`

Source: `src/api/urls.rs:23`

Responsibility

Extracts the last nonempty URL segment and removes any query string.

Signature and data contract

```
fn game_id(url: &str) -> &str
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn game_id(url: &str) -> &str {
    url.split('/')
        .filter(|part| !part.is_empty())
        .next_back()
        .unwrap_or_default()
        .split('?')
        .next()
        .unwrap_or_default()
}
```

Where it fits

The implementation lives in the focused `api::urls` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `validate_username`, `same_game_id`, `opening_from_eco`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 010 of 111: `api::urls::opening_from_eco`

Source: `src/api/urls.rs:33`

Responsibility

Builds a readable opening label from the final path segment of an ECO URL.

Signature and data contract

```
pub(super) fn opening_from_eco(url: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn opening_from_eco(url: &str) -> String {
    url.rsplit('/').next().unwrap_or(url).replace('-', " ")
}
```

Where it fits

The implementation lives in the focused `api::urls` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `validate_username`, `same_game_id`, `game_id`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 011 of 111: `chess::answers::normalize_answer`

Source: `src/chess/answers.rs:3`

Responsibility

Canonicalizes answer text by collapsing whitespace and lowercasing it.

Signature and data contract

```
pub fn normalize_answer(text: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn normalize_answer(text: &str) -> String {
    text.split_whitespace()
        .collect::<Vec<_>>()
        .join(" ")
        .to_lowercase()
}
```

Where it fits

The implementation lives in the focused `chess::answers` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `opening_matches`, `rating_tolerance`, `rating_is_close`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 012 of 111: `chess::answers::opening_matches`

Source: `src/chess/answers.rs:10`

Responsibility

Rejects blank guesses and compares normalized opening names.

Signature and data contract

```
pub fn opening_matches(guess: &str, expected: &str) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn opening_matches(guess: &str, expected: &str) -> bool {
    !normalize_answer(guess).is_empty() && normalize_answer(guess) == normalize_answer(expected)
}
```

Where it fits

The implementation lives in the focused `chess::answers` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `normalize_answer`, `rating_tolerance`, `rating_is_close`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 013 of 111: `chess::answers::rating_tolerance`

Source: `src/chess/answers.rs:14`

Responsibility

Maps Easy, Medium, and Hard difficulty names to their rating-error thresholds.

Signature and data contract

```
pub fn rating_tolerance(difficulty: &str) -> i32
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn rating_tolerance(difficulty: &str) -> i32 {
    match difficulty.to_ascii_lowercase().as_str() {
        "easy" => 200,
        "hard" => 50,
        _ => 100,
    }
}
```

Where it fits

The implementation lives in the focused `chess::answers` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `normalize_answer`, `opening_matches`, `rating_is_close`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 014 of 111: `chess::answers::rating_is_close`

Source: `src/chess/answers.rs:22`

Responsibility

Applies an inclusive absolute-error comparison using the selected tolerance.

Signature and data contract

```
pub fn rating_is_close(guess: i32, actual: i32, difficulty: &str) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn rating_is_close(guess: i32, actual: i32, difficulty: &str) -> bool {
    (guess - actual).abs() <= rating_tolerance(difficulty)
}
```

Where it fits

The implementation lives in the focused `chess::answers` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `normalize_answer`, `opening_matches`, `rating_tolerance`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 015 of 111: `chess::board::starting_position`

Source: `src/chess/board.rs:3`

Responsibility

Creates the row-major 64-square starting board used before the first ply.

Signature and data contract

```
pub(super) fn starting_position() -> [char; 64]
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn starting_position() -> [char; 64] {
    let rows = [
        "rnbqkbnr", "pppppppp", "........", "........", "........", "........", "PPPPPPPP",
        "RNBQKBNR",
    ];
    let mut board = ['.'; 64];
    for (rank, row) in rows.iter().enumerate() {
        for (file, piece) in row.chars().enumerate() {
            board[rank * 8 + file] = piece;
        }
    }
    board
}
```

Where it fits

The implementation lives in the focused `chess::board` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `piece_can_move`, `path_is_clear`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 016 of 111: `chess::board::piece_can_move`

Source: `src/chess/board.rs:17`

Responsibility

Checks movement shape, pawn direction, captures, and sliding-piece obstruction for one candidate origin.

Signature and data contract

```
pub(super) fn piece_can_move(
    board: &[char; 64],
    from_x: i32,
    from_y: i32,
    to_x: i32,
    to_y: i32,
    piece: char,
    white: bool,
    capture: bool,
) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn piece_can_move(
    board: &[char; 64],
    from_x: i32,
    from_y: i32,
    to_x: i32,
    to_y: i32,
    piece: char,
    white: bool,
    capture: bool,
) -> bool {
    let dx = to_x - from_x;
    let dy = to_y - from_y;
    if dx == 0 && dy == 0 {
        return false;
    }
    match piece {
        'N' => (dx.abs() == 1 && dy.abs() == 2) || (dx.abs() == 2 && dy.abs() == 1),
        'K' => dx.abs() <= 1 && dy.abs() <= 1,
        'R' if dx == 0 || dy == 0 => path_is_clear(board, from_x, from_y, to_x, to_y),
        'B' if dx.abs() == dy.abs() => path_is_clear(board, from_x, from_y, to_x, to_y),
        'Q' if dx == 0 || dy == 0 || dx.abs() == dy.abs() => {
            path_is_clear(board, from_x, from_y, to_x, to_y)
        }
        'P' => {
            let direction = if white { -1 } else { 1 };
            if capture {
                dx.abs() == 1 && dy == direction
            } else if dx != 0 {
                false
            } else if dy == direction {
                board[(to_y * 8 + to_x) as usize] == '.'
            } else {
                let start_rank = if white { 6 } else { 1 };
                dy == direction * 2
                    && from_y == start_rank
                    && board[((from_y + direction) * 8 + from_x) as usize] == '.'
                    && board[(to_y * 8 + to_x) as usize] == '.'
            }
        }
        _ => false,
    }
}
```

Where it fits

The implementation lives in the focused `chess::board` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `starting_position`, `path_is_clear`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 017 of 111: `chess::board::path_is_clear`

Source: `src/chess/board.rs:60`

Responsibility

Walks intermediate squares and rejects a sliding move when a piece blocks the path.

Signature and data contract

```
fn path_is_clear(board: &[char; 64], from_x: i32, from_y: i32, to_x: i32, to_y: i32) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn path_is_clear(board: &[char; 64], from_x: i32, from_y: i32, to_x: i32, to_y: i32) -> bool {
    let step_x = (to_x - from_x).signum();
    let step_y = (to_y - from_y).signum();
    let mut x = from_x + step_x;
    let mut y = from_y + step_y;
    while x != to_x || y != to_y {
        if board[(y * 8 + x) as usize] != '.' {
            return false;
        }
        x += step_x;
        y += step_y;
    }
    true
}
```

Where it fits

The implementation lives in the focused `chess::board` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `starting_position`, `piece_can_move`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 018 of 111: `chess::game::ChessGame::from_pgn`

Source: `src/chess/game.rs:27`

Responsibility

Provides the simple constructor when no API metadata overrides are available.

Signature and data contract

```
pub fn from_pgn(pgn: &str) -> Self
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn from_pgn(pgn: &str) -> Self {
        Self::with_metadata(pgn, None, None, None, None, None, false)
    }
```

Where it fits

The implementation lives in the focused `chess::game` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `with_metadata`, `demo`, `position_at`, `ply_count`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 019 of 111: `chess::game::ChessGame::with_metadata`

Source: `src/chess/game.rs:31`

Responsibility

Combines API overrides and PGN headers, applies accepted SAN moves, and stores an initial board plus each resulting snapshot.

Signature and data contract

```
pub fn with_metadata(
        pgn: &str,
        white_rating: Option<i32>,
        black_rating: Option<i32>,
        opening_override: Option<String>,
        eco_override: Option<String>,
        url_override: Option<String>,
        demo: bool,
    ) -> Self
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn with_metadata(
        pgn: &str,
        white_rating: Option<i32>,
        black_rating: Option<i32>,
        opening_override: Option<String>,
        eco_override: Option<String>,
        url_override: Option<String>,
        demo: bool,
    ) -> Self {
        let headers = pgn_headers(pgn);
        let mut board = starting_position();
        let mut positions = vec![board];
        let mut moves = Vec::new();
        let mut white_to_move = true;

        for token in pgn_move_tokens(pgn) {
            let san = strip_move_number(&token);
            if san.is_empty() || is_result(&san) || san.starts_with('$') || san == "e.p." {
                continue;
            }
            if apply_san(&mut board, &san, white_to_move) {
                moves.push(san);
                positions.push(board);
                white_to_move = !white_to_move;
            }
        }

        let white_name = header(&headers, "White").unwrap_or("White").to_owned();
        let black_name = header(&headers, "Black").unwrap_or("Black").to_owned();
        let opening = opening_override
            .filter(|value| !value.trim().is_empty())
            .or_else(|| header(&headers, "Opening").map(str::to_owned))
            .unwrap_or_else(|| "Unclassified opening".to_owned());
        let eco = eco_override
            .filter(|value| !value.trim().is_empty())
            .or_else(|| header(&headers, "ECO").map(str::to_owned))
            .unwrap_or_default();
        let source_url = url_override
            .filter(|value| !value.trim().is_empty())
            .or_else(|| header(&headers, "Site").map(str::to_owned))
            .unwrap_or_default();
```

```
        Self {
            white_name,
            black_name,
            white_rating: white_rating
                .or_else(|| header(&headers, "WhiteElo").and_then(parse_rating)),
            black_rating: black_rating
                .or_else(|| header(&headers, "BlackElo").and_then(parse_rating)),
            opening,
            eco,
            result: header(&headers, "Result").unwrap_or("*").to_owned(),
            time_control: header(&headers, "TimeControl")
                .unwrap_or("Unknown")
                .to_owned(),
            date: header(&headers, "UTCDate")
                .or_else(|| header(&headers, "Date"))
                .unwrap_or("Date unavailable")
                .to_owned(),
            source_url,
            demo,
            moves,
            positions,
        }
    }
```

Where it fits

The implementation lives in the focused `chess::game` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `from_pgn`, `demo`, `position_at`, `ply_count`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 020 of 111: `chess::game::ChessGame::demo`

Source: `src/chess/game.rs:97`

Responsibility

Creates the offline Italian Game fixture used by the app when no archive request is selected or available.

Signature and data contract

```
pub fn demo() -> Self
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn demo() -> Self {
        let pgn = r#"
[Event "ChessProject demo game"]
[Site "https://www.chess.com/"]
[Date "2026.10.14"]
[White "QuietKnight"]
[Black "TacticalTea"]
[Result "1-0"]
[WhiteElo "1180"]
[BlackElo "1432"]
[TimeControl "600+5"]
[ECO "C50"]
[Opening "Italian Game"]

1. e4 e5 2. Nf3 Nc6 3. Bc4 Bc5 4. c3 Nf6 5. d4 exd4 6. cxd4 Bb4+
7. Nc3 Nxe4 8. O-O Bxc3 9. d5 Na5 10. Bd3 f5 1-0
"#;
        let mut game = Self::from_pgn(pgn);
        game.demo = true;
        game
    }
```

Where it fits

The implementation lives in the focused `chess::game` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `from_pgn`, `with_metadata`, `position_at`, `ply_count`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 021 of 111: `chess::game::ChessGame::position_at`

Source: `src/chess/game.rs:119`

Responsibility

Returns a saved board snapshot while clamping an oversized ply to the final position.

Signature and data contract

```
pub fn position_at(&self, ply: usize) -> &[char; 64]
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn position_at(&self, ply: usize) -> &[char; 64] {
        let last = self.positions.len().saturating_sub(1);
        &self.positions[ply.min(last)]
    }
```

Where it fits

The implementation lives in the focused `chess::game` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `from_pgn`, `with_metadata`, `demo`, `ply_count`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 022 of 111: `chess::game::ChessGame::ply_count`

Source: `src/chess/game.rs:124`

Responsibility

Reports the number of board transitions without underflowing an empty snapshot list.

Signature and data contract

```
pub fn ply_count(&self) -> usize
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn ply_count(&self) -> usize {
        self.positions.len().saturating_sub(1)
    }
```

Where it fits

The implementation lives in the focused `chess::game` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `from_pgn`, `with_metadata`, `demo`, `position_at`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 023 of 111: `chess::pgn::pgn_headers`

Source: `src/chess/pgn.rs:5`

Responsibility

Reads bracketed PGN header records into a key/value map and unescapes quoted values.

Signature and data contract

```
pub fn pgn_headers(pgn: &str) -> HashMap<String, String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn pgn_headers(pgn: &str) -> HashMap<String, String> {
    let mut result = HashMap::new();
    for line in pgn.lines() {
        let line = line.trim();
        if !line.starts_with('[') || !line.ends_with(']') {
            continue;
        }
        let Some(space) = line.find(char::is_whitespace) else {
            continue;
        };
        let key = &line[1..space];
        let value = line[space..].trim().trim_end_matches(']').trim();
        let value = value
            .strip_prefix('"')
            .and_then(|text| text.strip_suffix('"'))
            .unwrap_or(value)
            .replace("\\\"", "\"");
        result.insert(key.to_owned(), value);
    }
    result
}
```

Where it fits

The implementation lives in the focused `chess::pgn` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `pgn_move_tokens`, `header`, `parse_rating`, `is_result`, `strip_move_number`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 024 of 111: `chess::pgn::pgn_move_tokens`

Source: `src/chess/pgn.rs:27`

Responsibility

Removes header lines, comments, and variations before returning whitespace-separated move tokens.

Signature and data contract

```
pub fn pgn_move_tokens(pgn: &str) -> Vec<String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn pgn_move_tokens(pgn: &str) -> Vec<String> {
    let mut body = String::new();
    let mut in_braces = false;
    let mut variation_depth = 0usize;
    let mut in_line_comment = false;

    for line in pgn.lines() {
        if line.trim().starts_with('[') {
            body.push(' ');
            continue;
        }
        for ch in line.chars() {
            if in_line_comment {
                continue;
            }
            if in_braces {
                if ch == '}' {
                    in_braces = false;
                    body.push(' ');
                }
                continue;
            }
            match ch {
                ';' if variation_depth == 0 => in_line_comment = true,
                '{' if variation_depth == 0 => in_braces = true,
                '(' => variation_depth += 1,
                ')' if variation_depth > 0 => variation_depth -= 1,
                _ if variation_depth == 0 => body.push(ch),
                _ => {}
            }
        }
        in_line_comment = false;
        body.push(' ');
    }

    body.split_whitespace().map(str::to_owned).collect()
}
```

Where it fits

The implementation lives in the focused `chess::pgn` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `pgn_headers`, `header`, `parse_rating`, `is_result`, `strip_move_number`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 025 of 111: `chess::pgn::header`

Source: `src/chess/pgn.rs:65`

Responsibility

Returns one borrowed metadata value without copying the header string.

Signature and data contract

```
pub(super) fn header<'a>(headers: &'a HashMap<String, String>, key: &str) -> Option<&'a str>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn header<'a>(headers: &'a HashMap<String, String>, key: &str) -> Option<&'a str> {
    headers.get(key).map(String::as_str)
}
```

Where it fits

The implementation lives in the focused `chess::pgn` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `pgn_headers`, `pgn_move_tokens`, `parse_rating`, `is_result`, `strip_move_number`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 026 of 111: `chess::pgn::parse_rating`

Source: `src/chess/pgn.rs:69`

Responsibility

Parses a positive integer rating and represents invalid or nonpositive input as None.

Signature and data contract

```
pub(super) fn parse_rating(text: &str) -> Option<i32>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn parse_rating(text: &str) -> Option<i32> {
    text.parse::<i32>().ok().filter(|rating| *rating > 0)
}
```

Where it fits

The implementation lives in the focused `chess::pgn` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `pgn_headers`, `pgn_move_tokens`, `header`, `is_result`, `strip_move_number`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 027 of 111: `chess::pgn::is_result`

Source: `src/chess/pgn.rs:73`

Responsibility

Recognizes the four PGN result markers so they are not mistaken for moves.

Signature and data contract

```
pub(super) fn is_result(token: &str) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn is_result(token: &str) -> bool {
    matches!(token, "1-0" | "0-1" | "1/2-1/2" | "*")
}
```

Where it fits

The implementation lives in the focused `chess::pgn` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `pgn_headers`, `pgn_move_tokens`, `header`, `parse_rating`, `strip_move_number`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 028 of 111: `chess::pgn::strip_move_number`

Source: `src/chess/pgn.rs:77`

Responsibility

Removes numeric and period prefixes such as 1. or 12... from a PGN token.

Signature and data contract

```
pub(super) fn strip_move_number(token: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn strip_move_number(token: &str) -> String {
    let mut index = 0;
    for byte in token.bytes() {
        if byte.is_ascii_digit() || byte == b'.' {
            index += 1;
        } else {
            break;
        }
    }
    token[index..].to_owned()
}
```

Where it fits

The implementation lives in the focused `chess::pgn` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `pgn_headers`, `pgn_move_tokens`, `header`, `parse_rating`, `is_result`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 029 of 111: `chess::san::apply_san`

Source: `src/chess/san.rs:5`

Responsibility

Interprets common SAN, finds a plausible origin square, mutates the board, and reports whether a move was applied.

Signature and data contract

```
pub(super) fn apply_san(board: &mut [char; 64], raw_san: &str, white_to_move: bool) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn apply_san(board: &mut [char; 64], raw_san: &str, white_to_move: bool) -> bool {
    let mut san = raw_san.to_owned();
    while san
        .chars()
        .last()
        .is_some_and(|character| matches!(character, '+' | '#' | '!' | '?'))
    {
        san.pop();
    }
    if san.is_empty() || san == "e.p." {
        return false;
    }

    if matches!(san.as_str(), "O-O" | "0-0" | "O-O-O" | "0-0-0") {
        let (king_from, king_to, rook_from, rook_to) = if white_to_move {
            if san.ends_with("O-O-O") || san.ends_with("0-0-0") {
                (60, 58, 56, 59)
            } else {
                (60, 62, 63, 61)
            }
        } else if san.ends_with("O-O-O") || san.ends_with("0-0-0") {
            (4, 2, 0, 3)
        } else {
            (4, 6, 7, 5)
        };
        board[king_to] = board[king_from];
        board[king_from] = '.';
        board[rook_to] = board[rook_from];
        board[rook_from] = '.';
        return true;
    }

    let promotion = san
        .split_once('=')
        .and_then(|(_, value)| value.chars().next());
    if let Some((before, _)) = san.split_once('=') {
        san = before.to_owned();
    }
    let capture = san.contains('x');
    let clean = san.replace('x', "");
    let bytes = clean.as_bytes();
    if bytes.len() < 2 {
        return false;
    }
    let dest_file = bytes[bytes.len() - 2];
    let dest_rank = bytes[bytes.len() - 1];
    if !(b'a'..=b'h').contains(&dest_file) || !(b'1'..=b'8').contains(&dest_rank) {
        return false;
    }
    let to_x = (dest_file - b'a') as i32;
    let to_y = 8 - (dest_rank - b'0') as i32;
    let prefix = &clean[..clean.len() - 2];
```

```
    let (piece, disambiguation) = match prefix.chars().next() {
        Some(piece @ ('K' | 'Q' | 'R' | 'B' | 'N')) => (piece, &prefix[1..]),
        _ => ('P', prefix),
    };
    let target = board[(to_y * 8 + to_x) as usize];
    if target != '.' && target.is_ascii_uppercase() == white_to_move {
        return false;
    }
    if (capture && target == '.' && piece != 'P') || (!capture && target != '.') {
        return false;
    }

    let mut origin = None;
    for from_y in 0..8i32 {
        for from_x in 0..8i32 {
            let index = (from_y * 8 + from_x) as usize;
            let candidate = board[index];
            if candidate == '.'
                || candidate.to_ascii_uppercase() != piece
                || candidate.is_ascii_uppercase() != white_to_move
                || !disambiguation_matches(disambiguation, from_x, from_y)
                || !piece_can_move(
                    board,
                    from_x,
                    from_y,
                    to_x,
                    to_y,
                    piece,
                    white_to_move,
                    capture,
                )
            {
                continue;
            }
            origin = Some((from_x, from_y));
            break;
        }
        if origin.is_some() {
            break;
        }
    }
    let Some((from_x, from_y)) = origin else {
        return false;
    };

    let from_index = (from_y * 8 + from_x) as usize;
    let to_index = (to_y * 8 + to_x) as usize;
    let mut moved_piece = board[from_index];
    if piece == 'P' && from_x != to_x && board[to_index] == '.' {
        let captured_y = if white_to_move { to_y + 1 } else { to_y - 1 };
        if (0..8).contains(&captured_y) {
            board[(captured_y * 8 + to_x) as usize] = '.';
        }
    }
    if piece == 'P' && (to_y == 0 || to_y == 7) {
        let promoted = promotion.unwrap_or('Q').to_ascii_uppercase();
        moved_piece = if white_to_move {
            promoted
        } else {
            promoted.to_ascii_lowercase()
        };
    }
    board[from_index] = '.';
    board[to_index] = moved_piece;
    true
}
```

Where it fits

The implementation lives in the focused `chess::san` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `disambiguation_matches`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 030 of 111: `chess::san::disambiguation_matches`

Source: `src/chess/san.rs:124`

Responsibility

Filters possible SAN origins by a specified source file or rank.

Signature and data contract

```
fn disambiguation_matches(disambiguation: &str, from_x: i32, from_y: i32) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn disambiguation_matches(disambiguation: &str, from_x: i32, from_y: i32) -> bool {
    disambiguation.chars().all(|ch| match ch {
        'a'..='h' => (ch as u8 - b'a') as i32 == from_x,
        '1'..='8' => 8 - (ch as u8 - b'0') as i32 == from_y,
        _ => true,
    })
}
```

Where it fits

The implementation lives in the focused `chess::san` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `apply_san`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 031 of 111: `cli::run`

Source: `src/cli.rs:26`

Responsibility

Owns the terminal lifecycle: raw mode, first check, input polling, redraws, and cleanup.

Signature and data contract

```
pub fn run()
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn run() {
    if let Err(error) = terminal::enable_raw_mode() {
        plain_fallback(&format!("Raw mode is unavailable ({error})."));
        return;
    }
    let mut stdout = CrLfWriter::new(io::stdout());
    if execute!(stdout, EnterAlternateScreen, Hide, Clear(ClearType::All)).is_err() {
        let _ = terminal::disable_raw_mode();
        plain_fallback("The terminal could not enter the interactive screen.");
        return;
    }

    let mut state = UiState::new();
    run_current_test(&mut state);
    let mut running = true;
    while running {
        draw(&mut stdout, &state);
        match event::poll(Duration::from_millis(200)) {
            Ok(true) => match event::read() {
                Ok(Event::Key(key)) => running = handle_key(key, &mut state),
                Ok(_) => {}
                Err(_) => running = false,
            },
            Ok(false) => {
                if matches!(state.screen, Screen::Home | Screen::Solution)
                    && source_modified(&state).is_some_and(|modified| {
                        state.last_modified.is_some_and(|last| modified > last)
                    })
                {
                    run_current_test(&mut state);
                }
            }
            Err(_) => running = false,
        }
    }

    let _ = execute!(stdout, Show, LeaveAlternateScreen);
    let _ = terminal::disable_raw_mode();
    println!(
        "We hope you're enjoying learning Rust! If you want to continue working on the exercises at a later point, you
           can simply run `gte` again in this directory."
    );
}
```

Where it fits

The implementation lives in the focused `cli` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 032 of 111: `cli::input::home::handle_home`

Source: `src/cli/input/home.rs:8`

Responsibility

Dispatches the home screen's navigation, hint, focused check, all-check, and Guidebook shortcuts.

Signature and data contract

```
pub(super) fn handle_home(key: KeyEvent, state: &mut UiState) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn handle_home(key: KeyEvent, state: &mut UiState) -> bool {
    match key.code {
        KeyCode::Char('q') | KeyCode::Char('Q') => return false,
        KeyCode::Char('p') | KeyCode::Char('P') => advance_previous(state),
        KeyCode::Char('n') | KeyCode::Char('N') => advance_next(state),
        KeyCode::Char('h') | KeyCode::Char('H') => state.show_hint = !state.show_hint,
        KeyCode::Char('l') | KeyCode::Char('L') => open_lesson_list(state),
        KeyCode::Char('c') | KeyCode::Char('C') => run_all_tests(state),
        KeyCode::Char('r') | KeyCode::Char('R') | KeyCode::Char('t') | KeyCode::Char('T') => {
            run_current_test(state);
        }
        KeyCode::Char('g') | KeyCode::Char('G') => {
            state.message =
                "Book: start `cargo run -- --web`, then open http://127.0.0.1:7878/guide".into();
        }
        _ => {}
    }
    true
}
```

Where it fits

The implementation lives in the focused `cli::input::home` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `handle_list`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 033 of 111: `cli::input::home::handle_list`

Source: `src/cli/input/home.rs:28`

Responsibility

Moves the exercise picker selection, opens the selected exercise, and returns to the home screen.

Signature and data contract

```
pub(super) fn handle_list(key: KeyEvent, state: &mut UiState) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn handle_list(key: KeyEvent, state: &mut UiState) -> bool {
    match key.code {
        KeyCode::Esc | KeyCode::Backspace | KeyCode::Char('l') | KeyCode::Char('L') => {
            state.screen = Screen::Home;
            state.message.clear();
        }
        KeyCode::Char('q') | KeyCode::Char('Q') => return false,
        KeyCode::Up | KeyCode::Char('p') | KeyCode::Char('P') => {
            state.selected = state.selected.saturating_sub(1);
        }
        KeyCode::Down | KeyCode::Char('n') | KeyCode::Char('N') => {
            state.selected = (state.selected + 1).min(LESSONS.len() - 1);
        }
        KeyCode::Enter => {
            state.lesson_index = state.selected;
            run_current_test(state);
        }
        KeyCode::Char('h') | KeyCode::Char('H') => state.show_hint = !state.show_hint,
        _ => {}
    }
    true
}
```

Where it fits

The implementation lives in the focused `cli::input::home` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `handle_home`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 034 of 111: `cli::input::lesson::handle_lesson`

Source: `src/cli/input/lesson.rs:8`

Responsibility

Handles one exercise's hint, concept check, solution, Guidebook, and navigation shortcuts.

Signature and data contract

```
pub(super) fn handle_lesson(key: KeyEvent, state: &mut UiState) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn handle_lesson(key: KeyEvent, state: &mut UiState) -> bool {
    match key.code {
        KeyCode::Esc | KeyCode::Backspace => state.screen = Screen::Home,
        KeyCode::Char('l') | KeyCode::Char('L') => open_lesson_list(state),
        KeyCode::Char('q') | KeyCode::Char('Q') => return false,
        KeyCode::Char('h') | KeyCode::Char('H') => state.show_hint = !state.show_hint,
        KeyCode::Char('n') | KeyCode::Char('N') => {
            advance_next(state);
        }
        KeyCode::Char('p') | KeyCode::Char('P') => {
            advance_previous(state);
        }
        KeyCode::Char('r') | KeyCode::Char('R') | KeyCode::Char('t') | KeyCode::Char('T') => {
            run_current_test(state);
        }
        KeyCode::Char('c') | KeyCode::Char('C') => {
            state.screen = Screen::Check;
            state.check_selected = 0;
            state.message = "Select an answer and press Enter to check your understanding.".into();
        }
        KeyCode::Char('s') | KeyCode::Char('S') => {
            let path = format!(
                "{}/{}",
                env!("CARGO_MANIFEST_DIR"),
                state.lesson().solution_file
            );
            let source = fs::read_to_string(path)
                .unwrap_or_else(|error| format!("Could not open reference solution: {error}"));
            state.solution = source.lines().map(str::to_owned).collect();
            state.solution_offset = 0;
            state.screen = Screen::Solution;
        }
        KeyCode::Char('g') | KeyCode::Char('G') => {
            state.message = format!(
                "Exercise reference: start `cargo run -- --web`, then open http://127.0.0.1:7878/guide/reference#{}",
                state.lesson().reference_anchor
            );
        }
        _ => {}
    }
    true
}
```

Where it fits

The implementation lives in the focused `cli::input::lesson` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `handle_check`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 035 of 111: `cli::input::lesson::handle_check`

Source: `src/cli/input/lesson.rs:51`

Responsibility

Selects and validates a concept-check answer while keeping exercise navigation available.

Signature and data contract

```
pub(super) fn handle_check(key: KeyEvent, state: &mut UiState) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn handle_check(key: KeyEvent, state: &mut UiState) -> bool {
    match key.code {
        KeyCode::Esc | KeyCode::Backspace => state.screen = Screen::Lesson,
        KeyCode::Char('l') | KeyCode::Char('L') => {
            open_lesson_list(state);
        }
        KeyCode::Char('n') | KeyCode::Char('N') => {
            advance_next(state);
        }
        KeyCode::Char('p') | KeyCode::Char('P') => {
            advance_previous(state);
        }
        KeyCode::Char('r') | KeyCode::Char('R') | KeyCode::Char('t') | KeyCode::Char('T') => {
            run_current_test(state)
        }
        KeyCode::Up => state.check_selected = state.check_selected.saturating_sub(1),
        KeyCode::Down => state.check_selected = (state.check_selected + 1).min(2),
        KeyCode::Char('1') => state.check_selected = 0,
        KeyCode::Char('2') => state.check_selected = 1,
        KeyCode::Char('3') => state.check_selected = 2,
        KeyCode::Enter => {
            if state.check_selected == state.lesson().answer {
                state.message =
                    "Concept check passed. Press R to run the Rust exercise check.".into();
                state.screen = Screen::Lesson;
            } else {
                state.message = "Not quite. Revisit the prompt or hint, then try again.".into();
            }
        }
        KeyCode::Char('q') | KeyCode::Char('Q') => return false,
        _ => {}
    }
    true
}
```

Where it fits

The implementation lives in the focused `cli::input::lesson` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `handle_lesson`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 036 of 111: `cli::input::handle_key`

Source: `src/cli/input/mod.rs:9`

Responsibility

Routes a key press according to the current screen and changes navigation or exercise state.

Signature and data contract

```
pub(super) fn handle_key(key: KeyEvent, state: &mut UiState) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn handle_key(key: KeyEvent, state: &mut UiState) -> bool {
    if matches!(key.code, KeyCode::Char('x') | KeyCode::Char('X'))
        && state.screen != Screen::ResetConfirm
    {
        if state.screen == Screen::List {
            state.lesson_index = state.selected;
        }
        state.screen = Screen::ResetConfirm;
        state.message.clear();
        return true;
    }
    if key.code == KeyCode::Char('?') {
        state.message = match state.screen {
            Screen::Home => "h:hint / l:list / c:check all / r:run / x:reset / n:next / p:previous / q:quit ?".into(),
            Screen::List => "n/p or ↑/↓ select / Enter: run exercise / l:back / q:quit".into(),
            Screen::Lesson => {
                "N/P next/previous · R run · H hint · C check · S solution · G Guidebook · L list · Q quit".into()
            }
            Screen::Check => "↑/↓ or 1–3 select · Enter check · R run · N/P next/previous · Esc/L lesson list · Q
              quit".into(),
            Screen::Solution => "↑/↓ or PgUp/PgDn scroll · N/P next/previous · Esc lesson · L list · Q quit".into(),
            Screen::TestResult => "R rerun · S solution · N/P next/previous · Esc lesson · L list · Q quit".into(),
            Screen::ResetConfirm => "y:reset current exercise / n:cancel".into(),
        };
        return true;
    }
    match state.screen {
        Screen::Home => home::handle_home(key, state),
        Screen::List => home::handle_list(key, state),
        Screen::Lesson => lesson::handle_lesson(key, state),
        Screen::Check => lesson::handle_check(key, state),
        Screen::Solution => results::handle_solution(key, state),
        Screen::TestResult => results::handle_test_result(key, state),
        Screen::ResetConfirm => results::handle_reset_confirm(key, state),
    }
}
```

Where it fits

The implementation lives in the focused `cli::input` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 037 of 111: `cli::input::results::handle_solution`

Source: `src/cli/input/results.rs:9`

Responsibility

Scrolls a reference solution and routes rerun, previous/next exercise, and screen navigation keys.

Signature and data contract

```
pub(super) fn handle_solution(key: KeyEvent, state: &mut UiState) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn handle_solution(key: KeyEvent, state: &mut UiState) -> bool {
    match key.code {
        KeyCode::Esc | KeyCode::Backspace => state.screen = Screen::Lesson,
        KeyCode::Char('r') | KeyCode::Char('R') | KeyCode::Char('t') | KeyCode::Char('T') => {
            run_current_test(state)
        }
        KeyCode::Char('l') | KeyCode::Char('L') => {
            open_lesson_list(state);
        }
        KeyCode::Char('n') | KeyCode::Char('N') => {
            advance_next(state);
        }
        KeyCode::Char('p') | KeyCode::Char('P') => {
            advance_previous(state);
        }
        KeyCode::Up => state.solution_offset = state.solution_offset.saturating_sub(1),
        KeyCode::Down => {
            state.solution_offset =
                (state.solution_offset + 1).min(state.solution.len().saturating_sub(1))
        }
        KeyCode::PageUp => state.solution_offset = state.solution_offset.saturating_sub(12),
        KeyCode::PageDown => {
            state.solution_offset =
                (state.solution_offset + 12).min(state.solution.len().saturating_sub(1))
        }
        KeyCode::Char('q') | KeyCode::Char('Q') => return false,
        _ => {}
    }
    true
}
```

Where it fits

The implementation lives in the focused `cli::input::results` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `handle_test_result`, `handle_reset_confirm`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 038 of 111: `cli::input::results::handle_test_result`

Source: `src/cli/input/results.rs:40`

Responsibility

Scrolls captured diagnostics and handles reruns, solution access, and exercise navigation.

Signature and data contract

```
pub(super) fn handle_test_result(key: KeyEvent, state: &mut UiState) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn handle_test_result(key: KeyEvent, state: &mut UiState) -> bool {
    match key.code {
        KeyCode::Esc | KeyCode::Backspace => state.screen = Screen::Lesson,
        KeyCode::Char('l') | KeyCode::Char('L') => {
            open_lesson_list(state);
        }
        KeyCode::Char('n') | KeyCode::Char('N') => {
            advance_next(state);
        }
        KeyCode::Char('p') | KeyCode::Char('P') => {
            advance_previous(state);
        }
        KeyCode::Char('r') | KeyCode::Char('R') | KeyCode::Char('t') | KeyCode::Char('T') => {
            run_current_test(state)
        }
        KeyCode::Char('s') | KeyCode::Char('S') => {
            let path = format!(
                "{}/{}",
                env!("CARGO_MANIFEST_DIR"),
                state.lesson().solution_file
            );
            let source = fs::read_to_string(path)
                .unwrap_or_else(|error| format!("Could not open reference solution: {error}"));
            state.solution = source.lines().map(str::to_owned).collect();
            state.solution_offset = 0;
            state.screen = Screen::Solution;
        }
        KeyCode::Up => state.solution_offset = state.solution_offset.saturating_sub(1),
        KeyCode::Down => {
            state.solution_offset =
                (state.solution_offset + 1).min(state.test_output.len().saturating_sub(1))
        }
        KeyCode::Char('q') | KeyCode::Char('Q') => return false,
        _ => {}
    }
    true
}
```

Where it fits

The implementation lives in the focused `cli::input::results` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `handle_solution`, `handle_reset_confirm`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 039 of 111: `cli::input::results::handle_reset_confirm`

Source: `src/cli/input/results.rs:78`

Responsibility

Accepts a source reset, cancels back to the home screen, or exits the terminal UI.

Signature and data contract

```
pub(super) fn handle_reset_confirm(key: KeyEvent, state: &mut UiState) -> bool
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn handle_reset_confirm(key: KeyEvent, state: &mut UiState) -> bool {
    match key.code {
        KeyCode::Char('y') | KeyCode::Char('Y') => reset_current_exercise(state),
        KeyCode::Char('n') | KeyCode::Char('N') | KeyCode::Esc | KeyCode::Backspace => {
            state.screen = Screen::Home;
            state.message = "Reset cancelled.".into();
        }
        KeyCode::Char('q') | KeyCode::Char('Q') => return false,
        _ => {}
    }
    true
}
```

Where it fits

The implementation lives in the focused `cli::input::results` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `handle_solution`, `handle_test_result`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 040 of 111: `cli::navigation::open_lesson_list`

Source: `src/cli/navigation.rs:6`

Responsibility

Copies the active lesson into the picker selection and opens the list screen.

Signature and data contract

```
pub(super) fn open_lesson_list(state: &mut UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn open_lesson_list(state: &mut UiState) {
    state.selected = state.lesson_index;
    state.screen = Screen::List;
    state.message.clear();
}
```

Where it fits

The implementation lives in the focused `cli::navigation` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `advance_next`, `advance_previous`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 041 of 111: `cli::navigation::advance_next`

Source: `src/cli/navigation.rs:12`

Responsibility

Blocks forward navigation until the current exercise passes, then selects and runs the next incomplete lesson.

Signature and data contract

```
pub(super) fn advance_next(state: &mut UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn advance_next(state: &mut UiState) {
    if !state.completed.contains(state.lesson().id) {
        state.message =
            "This exercise is not complete yet. Edit it, then press R to run it.".into();
        return;
    }
    let next = (state.lesson_index + 1..LESSONS.len())
        .chain(0..state.lesson_index)
        .find(|index| !state.completed.contains(LESSONS[*index].id));
    match next {
        Some(index) => {
            state.lesson_index = index;
            state.selected = index;
            state.show_hint = false;
            state.message.clear();
            run_current_test(state);
        }
        None => {
            state.message = "All exercises are complete. Use L to review the exercise list.".into();
        }
    }
}
```

Where it fits

The implementation lives in the focused `cli::navigation` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `open_lesson_list`, `advance_previous`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 042 of 111: `cli::navigation::advance_previous`

Source: `src/cli/navigation.rs:35`

Responsibility

Moves one lesson backward and immediately checks that lesson.

Signature and data contract

```
pub(super) fn advance_previous(state: &mut UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn advance_previous(state: &mut UiState) {
    if state.lesson_index == 0 {
        state.message = "This is the first exercise.".into();
        return;
    }
    state.lesson_index -= 1;
    state.selected = state.lesson_index;
    state.show_hint = false;
    state.message.clear();
    run_current_test(state);
}
```

Where it fits

The implementation lives in the focused `cli::navigation` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `open_lesson_list`, `advance_next`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 043 of 111: `cli::progress::progress_path`

Source: `src/cli/progress.rs:5`

Responsibility

Builds the path to the repository-local ignored completion file.

Signature and data contract

```
fn progress_path() -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn progress_path() -> String {
    format!("{}/.gte-state.txt", env!("CARGO_MANIFEST_DIR"))
}
```

Where it fits

The implementation lives in the focused `cli::progress` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `load_progress`, `save_progress`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 044 of 111: `cli::progress::load_progress`

Source: `src/cli/progress.rs:9`

Responsibility

Reads saved ids and keeps only ids that still exist in the current lesson catalog.

Signature and data contract

```
pub(super) fn load_progress() -> HashSet<&'static str>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn load_progress() -> HashSet<&'static str> {
    let Ok(saved) = fs::read_to_string(progress_path()) else {
        return HashSet::new();
    };
    LESSONS
        .iter()
        .filter(|lesson| saved.lines().any(|line| line == lesson.id))
        .map(|lesson| lesson.id)
        .collect()
}
```

Where it fits

The implementation lives in the focused `cli::progress` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `progress_path`, `save_progress`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 045 of 111: `cli::progress::save_progress`

Source: `src/cli/progress.rs:20`

Responsibility

Writes completed ids in lesson order to make the progress file stable and readable.

Signature and data contract

```
pub(super) fn save_progress(completed: &HashSet<&'static str>) -> io::Result<()>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn save_progress(completed: &HashSet<&'static str>) -> io::Result<()> {
    let saved = LESSONS
        .iter()
        .filter(|lesson| completed.contains(lesson.id))
        .map(|lesson| lesson.id)
        .collect::<Vec<_>>()
        .join("\n");
    fs::write(progress_path(), saved)
}
```

Where it fits

The implementation lives in the focused `cli::progress` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `progress_path`, `load_progress`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 046 of 111: `cli::render::check::draw_check`

Source: `src/cli/render/check.rs:7`

Responsibility

Renders the concept-check choices and highlights the currently selected answer.

Signature and data contract

```
pub(super) fn draw_check<W: Write>(stdout: &mut W, state: &UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn draw_check<W: Write>(stdout: &mut W, state: &UiState) {
    let lesson = state.lesson();
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::Green),
        Print("KNOWLEDGE CHECK\n"),
        ResetColor
    );
    let _ = queue!(
        stdout,
        Print(format!("{}\n\n{}\n", lesson.title, lesson.question))
    );
    for (index, choice) in lesson.choices.iter().enumerate() {
        let pointer = if index == state.check_selected {
            "›"
        } else {
            " "
        };
        if index == state.check_selected {
            let _ = queue!(stdout, SetForegroundColor(Color::Green));
        }
        let _ = queue!(
            stdout,
            Print(format!(" {pointer} {}. {choice}\n", index + 1))
        );
        let _ = queue!(stdout, ResetColor);
    }
    let _ = queue!(
        stdout,
        Print(
            "\n↑/↓ or 1–3 select · Enter check · R run · N/P next/previous · L list · Esc lesson\n"
        )
    );
}
```

Where it fits

The implementation lives in the focused `cli::render::check` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 047 of 111: `cli::render::home::draw_home`

Source: `src/cli/render/home.rs:9`

Responsibility

Shows progress, the current task, optional hint, and a terminal-height-limited slice of check output.

Signature and data contract

```
pub(super) fn draw_home<W: Write>(stdout: &mut W, state: &UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn draw_home<W: Write>(stdout: &mut W, state: &UiState) {
    let lesson = state.lesson();
    let bar_width = terminal::size()
        .map(|(columns, _)| (columns as usize).saturating_sub(28).clamp(10, 94))
        .unwrap_or(52);
    let complete_count = state.completed.len().min(LESSONS.len());
    let filled = complete_count * bar_width / LESSONS.len();
    let marker_width = if complete_count < LESSONS.len() { 1 } else { 0 };
    let exercise_file = lesson.id.replace('-', "_");
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::White),
        Print("Progress: ["),
        SetForegroundColor(Color::Green),
        Print("#".repeat(filled)),
        SetForegroundColor(Color::Yellow),
        Print(if complete_count < LESSONS.len() {
            ">"
        } else {
            ""
        }),
        SetForegroundColor(Color::DarkGrey),
        Print("-".repeat(bar_width.saturating_sub(filled + marker_width))),
        SetForegroundColor(Color::White),
        Print(format!("] {complete_count}/{}\n", LESSONS.len())),
        ResetColor,
        SetForegroundColor(Color::DarkGrey),
        Print(format!(
            "Current exercise: exercises/src/{exercise_file}.rs\n\n"
        )),
        ResetColor,
        SetForegroundColor(Color::White),
        Print(format!("{}\n{}\n", lesson.title, lesson.prompt)),
        ResetColor
    );
    if state.show_hint {
        let _ = queue!(
            stdout,
            SetForegroundColor(Color::Yellow),
            Print(format!("\nHINT\n{}\n", lesson.hint)),
            ResetColor
        );
    }
    let rows = terminal::size()
        .map(|(_, rows)| rows as usize)
        .unwrap_or(24);
    let output_lines = state.test_output.len().min(rows.saturating_sub(12));
    if !state.test_output.is_empty() {
        let _ = queue!(
            stdout,
            SetForegroundColor(if state.test_passed {
                Color::Green
```

```
            } else {
                Color::Yellow
            }),
            Print(if state.test_passed {
                "\n✓ Exercise check passed\n"
            } else {
                "\nExercise check output\n"
            }),
            ResetColor
        );
        for line in state
            .test_output
            .iter()
            .skip(state.test_output.len().saturating_sub(output_lines))
        {
            let _ = queue!(stdout, Print(format!("{line}\n")));
        }
        if output_lines < state.test_output.len() {
            let _ = queue!(
                stdout,
                SetForegroundColor(Color::DarkGrey),
                Print("… output shortened to fit terminal\n"),
                ResetColor
            );
        }
    }
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::DarkGrey),
        Print(if state.completed.contains(lesson.id) {
            "\nh:hint / l:list / c:check all / x:reset / n:next / q:quit ?\n"
        } else {
            "\nh:hint / l:list / c:check all / x:reset / q:quit ?\n"
        }),
        ResetColor
    );
}
```

Where it fits

The implementation lives in the focused `cli::render::home` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 048 of 111: `cli::render::lesson::draw_lesson`

Source: `src/cli/render/lesson.rs:7`

Responsibility

Displays a lesson prompt, code location, NOTE/TODO guidance, and available actions.

Signature and data contract

```
pub(super) fn draw_lesson<W: Write>(stdout: &mut W, state: &UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn draw_lesson<W: Write>(stdout: &mut W, state: &UiState) {
    let lesson = state.lesson();
    let exercise_file = lesson.id.replace('-', "_");
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::Green),
        Print(format!("{}\n", lesson.track)),
        ResetColor
    );
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::White),
        Print(format!("{}\n\n", lesson.title)),
        ResetColor
    );
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::DarkGrey),
        Print(format!(
            "EDIT exercises/src/{exercise_file}.rs\nREFERENCE exercises/reference.md#{}\n\n",
            lesson.reference_anchor
        )),
        ResetColor
    );
    let _ = queue!(
        stdout,
        Print(format!("{}\n\nTASK\n{}\n", lesson.concept, lesson.prompt))
    );
    if state.show_hint {
        let _ = queue!(
            stdout,
            SetForegroundColor(Color::Yellow),
            Print(format!("\nHINT\n{}\n", lesson.hint)),
            ResetColor
        );
    }
    if state.completed.contains(lesson.id) {
        let _ = queue!(
            stdout,
            SetForegroundColor(Color::Green),
            Print("\n✓ Rust exercise tests passed\n"),
            ResetColor
        );
    }
    let _ = queue!(
        stdout,
        Print(
            "\nN next · P previous · R run check · H hint · C concept check · S solution · G Guidebook · L lesson
              list\n"
        )
    );
}
```

Where it fits

The implementation lives in the focused `cli::render::lesson` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 049 of 111: `cli::render::list::draw_list`

Source: `src/cli/render/list.rs:8`

Responsibility

Renders all exercises with status markers and the keyboard controls for selecting one.

Signature and data contract

```
pub(super) fn draw_list<W: Write>(stdout: &mut W, state: &UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn draw_list<W: Write>(stdout: &mut W, state: &UiState) {
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::White),
        Print("EXERCISE LIST\n"),
        ResetColor
    );
    for (index, lesson) in LESSONS.iter().enumerate() {
        let pointer = if index == state.selected { "›" } else { " " };
        let status = if state.completed.contains(lesson.id) {
            "✓"
        } else if state.started.contains(lesson.id) {
            "◉"
        } else {
            "○"
        };
        if index == state.selected {
            let _ = queue!(stdout, SetForegroundColor(Color::Green));
        }
        let _ = queue!(
            stdout,
            Print(format!(
                " {pointer} {status} {:<28} {}\n",
                lesson.title, lesson.track
            ))
        );
        let _ = queue!(stdout, ResetColor);
    }
    let _ = queue!(
        stdout,
        Print(
            "\n○ not started · ◉ in progress · ✓ complete\nN/P or ↑/↓ select · Enter open · L/Esc return\n"
        )
    );
}
```

Where it fits

The implementation lives in the focused `cli::render::list` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 050 of 111: `cli::render::draw`

Source: `src/cli/render/mod.rs:18`

Responsibility

Clears the frame, paints the GTE terminal wordmark, and dispatches to the active screen renderer.

Signature and data contract

```
pub(super) fn draw<W: Write>(stdout: &mut W, state: &UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn draw<W: Write>(stdout: &mut W, state: &UiState) {
    let _ = queue!(stdout, MoveTo(0, 0), Clear(ClearType::All));
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::Green),
        Print(
            " ████    ██████   ██████\n██        ██        ██\n██ ███    ████     ████\n██  ██    ██        ██\n
              ████    ██████   ██████\n\n  GUESS THE ELO · RUST CODEBASE WALKTHROUGH\n\n"
        )
    );
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::DarkGrey),
        Print(" Rustlings-style project exercises\n\n"),
        ResetColor
    );

    match state.screen {
        Screen::Home => home::draw_home(stdout, state),
        Screen::List => list::draw_list(stdout, state),
        Screen::Lesson => lesson::draw_lesson(stdout, state),
        Screen::Check => check::draw_check(stdout, state),
        Screen::Solution => solution::draw_solution(stdout, state),
        Screen::TestResult => test_result::draw_test_result(stdout, state),
        Screen::ResetConfirm => reset_confirm::draw_reset_confirm(stdout, state),
    }

    if !state.message.is_empty() {
        let _ = queue!(stdout, Print(format!("\n{}\n", state.message)));
    }
    if state.screen != Screen::Home {
        let _ = queue!(
            stdout,
            SetForegroundColor(Color::DarkGrey),
            Print("\nQ quit · Esc back · ? shortcuts"),
            ResetColor
        );
    }
    let _ = stdout.flush();
}
```

Where it fits

The implementation lives in the focused `cli::render` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `plain_fallback`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 051 of 111: `cli::render::plain_fallback`

Source: `src/cli/render/mod.rs:58`

Responsibility

Prints a noninteractive lesson summary when terminal raw mode or screen setup is unavailable.

Signature and data contract

```
pub(super) fn plain_fallback(reason: &str)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn plain_fallback(reason: &str) {
    println!("ChessProject interactive lesson UI\n{reason}\n");
    for (index, lesson) in LESSONS.iter().enumerate() {
        println!("{}. {} — {}", index + 1, lesson.title, lesson.concept);
    }
    println!(
        "\nStart the browser Guidebook with `cargo run -- --web` at http://127.0.0.1:7878/guide."
    );
}
```

Where it fits

The implementation lives in the focused `cli::render` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `draw`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 052 of 111: `cli::render::reset_confirm::draw_reset_confirm`

Source: `src/cli/render/reset_confirm.rs:7`

Responsibility

Explains which source file will be restored and where its backup will be written.

Signature and data contract

```
pub(super) fn draw_reset_confirm<W: Write>(stdout: &mut W, state: &UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn draw_reset_confirm<W: Write>(stdout: &mut W, state: &UiState) {
    let path = state.lesson().id.replace('-', "_");
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::Yellow),
        Print("Reset current exercise?\n\n"),
        ResetColor,
        Print(format!(
            "This restores exercises/src/{path}.rs from Git HEAD.\n"
        )),
        Print("A copy of the current file will be saved in .gte-backup/.\n\n"),
        SetForegroundColor(Color::White),
        Print("Reset (y/n)? "),
        ResetColor
    );
}
```

Where it fits

The implementation lives in the focused `cli::render::reset_confirm` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 053 of 111: `cli::render::solution::draw_solution`

Source: `src/cli/render/solution.rs:8`

Responsibility

Shows test names and the final test summary before the matching solution source.

Signature and data contract

```
pub(super) fn draw_solution<W: Write>(stdout: &mut W, state: &UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn draw_solution<W: Write>(stdout: &mut W, state: &UiState) {
    let complete_count = state.completed.len().min(LESSONS.len());
    let exercise_file = state.lesson().id.replace('-', "_");
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::Green),
        Print(format!("REFERENCE · {}\n", state.lesson().title)),
        ResetColor,
        SetForegroundColor(Color::DarkGrey),
        Print(format!(
            "Progress: {complete_count}/{} · exercises/src/{exercise_file}.rs\n\n",
            LESSONS.len()
        )),
        ResetColor
    );
    if state.completed.contains(state.lesson().id) {
        let _ = queue!(
            stdout,
            SetForegroundColor(Color::Green),
            Print("✓ Exercise tests passed · matching solution loaded\n"),
            ResetColor
        );
        let test_lines = state
            .test_output
            .iter()
            .filter(|line| {
                line.starts_with("running ")
                    || line.starts_with("test ")
                    || line.starts_with("test result:")
            })
            .take(7);
        for line in test_lines {
            let _ = queue!(stdout, Print(format!("  {line}\n")));
        }
        let _ = queue!(stdout, Print("\n"));
    }
    let _ = queue!(
        stdout,
        SetForegroundColor(Color::Yellow),
        Print("MATCHING REFERENCE SOLUTION\n"),
        ResetColor
    );
    for line in state.solution.iter().skip(state.solution_offset).take(20) {
        let _ = queue!(stdout, Print(format!("{line}\n")));
    }
    let _ = queue!(
        stdout,
        Print("\n↑/↓ scroll · PgUp/PgDn page · N/P next/previous · L lesson list · Esc return\n")
    );
}
```

Where it fits

The implementation lives in the focused `cli::render::solution` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 054 of 111: `cli::render::test_result::draw_test_result`

Source: `src/cli/render/test_result.rs:7`

Responsibility

Displays the test output for a focused or all-target run and its pass/fail label.

Signature and data contract

```
pub(super) fn draw_test_result<W: Write>(stdout: &mut W, state: &UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn draw_test_result<W: Write>(stdout: &mut W, state: &UiState) {
    let (label, color) = if state.test_passed {
        ("TESTS PASSED", Color::Green)
    } else {
        ("TESTS NEED WORK", Color::Yellow)
    };
    let _ = queue!(
        stdout,
        SetForegroundColor(color),
        Print(format!("{label} · {}\n", state.lesson().title)),
        ResetColor
    );
    for line in state
        .test_output
        .iter()
        .skip(state.solution_offset)
        .take(20)
    {
        let _ = queue!(stdout, Print(format!("{line}\n")));
    }
    let _ = queue!(
        stdout,
        Print("\nR rerun · S solution · N/P next/previous · L lesson list · Esc return\n")
    );
}
```

Where it fits

The implementation lives in the focused `cli::render::test_result` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 055 of 111: `cli::reset::reset_current_exercise`

Source: `src/cli/reset.rs:10`

Responsibility

Backs up the active source, restores its Git HEAD version, clears its progress, and reruns it.

Signature and data contract

```
pub(super) fn reset_current_exercise(state: &mut UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn reset_current_exercise(state: &mut UiState) {
    let relative = format!("exercises/src/{}.rs", state.lesson().id.replace('-', "_"));
    let source_path = source_path(state);
    let backup_path = PathBuf::from(env!("CARGO_MANIFEST_DIR"))
        .join(".gte-backup")
        .join(state.lesson().id.replace('-', "_") + ".rs");
    let original = Command::new("git")
        .args(["show", &format!("HEAD:{relative}")])
        .current_dir(env!("CARGO_MANIFEST_DIR"))
        .output();
    let original = match original {
        Ok(output) if output.status.success() => output.stdout,
        Ok(output) => {
            state.screen = Screen::Home;
            state.message = format!(
                "Could not restore the exercise from Git: {}",
                String::from_utf8_lossy(&output.stderr).trim()
            );
            return;
        }
        Err(error) => {
            state.screen = Screen::Home;
            state.message = format!("Could not start Git to reset the exercise: {error}");
            return;
        }
    };
    if let Err(error) = fs::create_dir_all(backup_path.parent().expect("backup parent"))
        .and_then(|()| fs::copy(&source_path, &backup_path).map(|_| ()))
        .and_then(|()| fs::write(&source_path, original))
    {
        state.screen = Screen::Home;
        state.message =
            format!("Reset failed; the current file is backed up when possible: {error}");
        return;
    }
    state.completed.remove(state.lesson().id);
    state.started.remove(state.lesson().id);
    let _ = save_progress(&state.completed);
    state.screen = Screen::Home;
    state.show_hint = false;
    state.message = format!(
        "Reset {}. Previous contents saved in .gte-backup/.",
        state.lesson().title
    );
    run_current_test(state);
}
```

Where it fits

The implementation lives in the focused `cli::reset` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 056 of 111: `cli::runner::run_current_test`

Source: `src/cli/runner.rs:9`

Responsibility

Runs one lesson target, updates progress, stores diagnostics, and loads that lesson's matching solution on success.

Signature and data contract

```
pub(super) fn run_current_test(state: &mut UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn run_current_test(state: &mut UiState) {
    let lesson = state.lesson();
    state.started.insert(lesson.id);
    let (passed, text) = run_lesson_test(lesson);
    state.last_modified = source_modified(state);
    state.test_passed = passed;
    state.test_output = text.lines().map(str::to_owned).collect();
    state.solution_offset = 0;
    if passed {
        state.completed.insert(lesson.id);
        let _ = save_progress(&state.completed);
        let path = format!("{}/{}", env!("CARGO_MANIFEST_DIR"), lesson.solution_file);
        match fs::read_to_string(path) {
            Ok(source) => {
                state.solution = source.lines().map(str::to_owned).collect();
                state.screen = Screen::Solution;
                state.message =
                    "Exercise check passed. The matching reference solution loaded automatically."
                        .into();
            }
            Err(error) => {
                state.screen = Screen::TestResult;
                state.message = format!(
                    "Exercise check passed, but the reference solution could not load: {error}"
                );
            }
        }
    } else {
        state.completed.remove(lesson.id);
        let _ = save_progress(&state.completed);
        state.screen = Screen::Home;
        state.message = "Edit the exercise code, check the hint, and run the check again.".into();
    }
}
```

Where it fits

The implementation lives in the focused `cli::runner` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `run_all_tests`, `run_lesson_test`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 057 of 111: `cli::runner::run_all_tests`

Source: `src/cli/runner.rs:44`

Responsibility

Runs every registered exercise target and saves each pass/fail completion result.

Signature and data contract

```
pub(super) fn run_all_tests(state: &mut UiState)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn run_all_tests(state: &mut UiState) {
    let mut summary = Vec::new();
    let mut passed_count = 0;
    for lesson in &LESSONS {
        state.started.insert(lesson.id);
        let (passed, output) = run_lesson_test(lesson);
        if passed {
            state.completed.insert(lesson.id);
            passed_count += 1;
            summary.push(format!("✓ {}", lesson.title));
        } else {
            state.completed.remove(lesson.id);
            summary.push(format!("✗ {}", lesson.title));
            summary.extend(
                output
                    .lines()
                    .rev()
                    .take(5)
                    .collect::<Vec<_>>()
                    .into_iter()
                    .rev()
                    .map(|line| format!("  {line}")),
            );
        }
    }
    let _ = save_progress(&state.completed);
    state.test_passed = passed_count == LESSONS.len();
    state.test_output = summary;
    state.solution_offset = 0;
    state.screen = Screen::TestResult;
    state.message = format!(
        "Check all finished: {passed_count}/{} exercises passed.",
        LESSONS.len()
    );
}
```

Where it fits

The implementation lives in the focused `cli::runner` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `run_current_test`, `run_lesson_test`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 058 of 111: `cli::runner::run_lesson_test`

Source: `src/cli/runner.rs:80`

Responsibility

Invokes Cargo with one integration-test target and captures both stdout and stderr.

Signature and data contract

```
fn run_lesson_test(lesson: &Lesson) -> (bool, String)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn run_lesson_test(lesson: &Lesson) -> (bool, String) {
    let manifest = format!("{}/exercises/Cargo.toml", env!("CARGO_MANIFEST_DIR"));
    let target = lesson.id.replace('-', "_");
    let output = Command::new("cargo")
        .arg("test")
        .arg("--manifest-path")
        .arg(&manifest)
        .arg("--test")
        .arg(&target)
        .current_dir(env!("CARGO_MANIFEST_DIR"))
        .output();
    match output {
        Ok(output) => {
            let mut text = String::from_utf8_lossy(&output.stdout).into_owned();
            text.push_str(&String::from_utf8_lossy(&output.stderr));
            (output.status.success(), text)
        }
        Err(error) => (false, format!("Could not start Cargo: {error}")),
    }
}
```

Where it fits

The implementation lives in the focused `cli::runner` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `run_current_test`, `run_all_tests`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 059 of 111: `cli::source::source_path`

Source: `src/cli/source.rs:7`

Responsibility

Maps a lesson id to its learner-editable Rust file under exercises/src.

Signature and data contract

```
pub(super) fn source_path(state: &UiState) -> PathBuf
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn source_path(state: &UiState) -> PathBuf {
    let file_name = state.lesson().id.replace('-', "_");
    PathBuf::from(env!("CARGO_MANIFEST_DIR"))
        .join("exercises/src")
        .join(format!("{file_name}.rs"))
}
```

Where it fits

The implementation lives in the focused `cli::source` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `source_modified`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 060 of 111: `cli::source::source_modified`

Source: `src/cli/source.rs:14`

Responsibility

Reads the active exercise file's modification time for automatic check reruns.

Signature and data contract

```
pub(super) fn source_modified(state: &UiState) -> Option<SystemTime>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn source_modified(state: &UiState) -> Option<SystemTime> {
    fs::metadata(source_path(state)).ok()?.modified().ok()
}
```

Where it fits

The implementation lives in the focused `cli::source` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `source_path`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 061 of 111: `cli::state::UiState::new`

Source: `src/cli/state.rs:35`

Responsibility

Loads saved lesson completion and selects the first exercise that still needs work.

Signature and data contract

```
pub(super) fn new() -> Self
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn new() -> Self {
        let completed = load_progress();
        let lesson_index = LESSONS
            .iter()
            .position(|lesson| !completed.contains(lesson.id))
            .unwrap_or(0);
        Self {
            screen: Screen::Home,
            selected: lesson_index,
            lesson_index,
            check_selected: 0,
            show_hint: false,
            started: completed.clone(),
            completed,
            message: String::new(),
            solution: Vec::new(),
            solution_offset: 0,
            test_output: Vec::new(),
            test_passed: false,
            last_modified: None,
        }
    }
```

Where it fits

The implementation lives in the focused `cli::state` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `lesson`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 062 of 111: `cli::state::UiState::lesson`

Source: `src/cli/state.rs:58`

Responsibility

Looks up the static lesson currently selected by the UI state.

Signature and data contract

```
pub(super) fn lesson(&self) -> &'static Lesson
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn lesson(&self) -> &'static Lesson {
        &LESSONS[self.lesson_index]
    }
```

Where it fits

The implementation lives in the focused `cli::state` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `new`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 063 of 111: `cli::writer::CrLfWriter<W>::new`

Source: `src/cli/writer.rs:11`

Responsibility

Initializes the writer adapter that keeps track of whether the last byte was a carriage return.

Signature and data contract

```
pub(super) fn new(inner: W) -> Self
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn new(inner: W) -> Self {
        Self {
            inner,
            last_was_carriage_return: false,
        }
    }
```

Where it fits

The implementation lives in the focused `cli::writer` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `write`, `flush`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 064 of 111: `cli::writer::<CrLfWriter<W> as Write>::write`

Source: `src/cli/writer.rs:20`

Responsibility

Adapts line-feed bytes for raw-mode terminals so output lines return to column one.

Signature and data contract

```
fn write(&mut self, bytes: &[u8]) -> io::Result<usize>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn write(&mut self, bytes: &[u8]) -> io::Result<usize> {
        let mut normalized = Vec::with_capacity(bytes.len() + 8);
        for &byte in bytes {
            if byte == b'\n' && !self.last_was_carriage_return {
                normalized.push(b'\r');
            }
            normalized.push(byte);
            self.last_was_carriage_return = byte == b'\r';
        }
        self.inner.write_all(&normalized)?;
        Ok(bytes.len())
    }
```

Where it fits

The implementation lives in the focused `cli::writer` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `new`, `flush`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 065 of 111: `cli::writer::<CrLfWriter<W> as Write>::flush`

Source: `src/cli/writer.rs:33`

Responsibility

Forwards an explicit flush to the wrapped terminal writer.

Signature and data contract

```
fn flush(&mut self) -> io::Result<()>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn flush(&mut self) -> io::Result<()> {
        self.inner.flush()
    }
```

Where it fits

The implementation lives in the focused `cli::writer` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `new`, `write`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 066 of 111: `lessons::model::lesson`

Source: `src/lessons/model.rs:18`

Responsibility

Constructs one static curriculum record that joins its prompt, test, solution, difficulty, and codebase reference.

Signature and data contract

```
pub(super) const fn lesson(
    id: &'static str,
    track: &'static str,
    title: &'static str,
    concept: &'static str,
    prompt: &'static str,
    hint: &'static str,
    question: &'static str,
    choices: [&'static str; 3],
    answer: usize,
    solution_file: &'static str,
    reference_anchor: &'static str,
) -> Lesson
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) const fn lesson(
    id: &'static str,
    track: &'static str,
    title: &'static str,
    concept: &'static str,
    prompt: &'static str,
    hint: &'static str,
    question: &'static str,
    choices: [&'static str; 3],
    answer: usize,
    solution_file: &'static str,
    reference_anchor: &'static str,
) -> Lesson {
    Lesson {
        id,
        track,
        title,
        concept,
        prompt,
        hint,
        question,
        choices,
        answer,
        solution_file,
        reference_anchor,
    }
}
```

Where it fits

The implementation lives in the focused `lessons::model` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 067 of 111: `web::book::catalog::book_chapters`

Source: `src/web/book/catalog.rs:75`

Responsibility

Builds the complete Guidebook catalog from its stable chapter files and the generated function-page source.

Signature and data contract

```
pub(in crate::web) fn book_chapters() -> &'static [BookChapter]
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn book_chapters() -> &'static [BookChapter] {
    static CHAPTERS: OnceLock<Vec<BookChapter>> = OnceLock::new();
    CHAPTERS.get_or_init(|| {
        let mut chapters: Vec<BookChapter> = BASE_BOOK_CHAPTERS
            .iter()
            .map(|(slug, title, source)| BookChapter {
                slug: (*slug).to_owned(),
                title: (*title).to_owned(),
                source: (*source).to_owned(),
            })
            .collect();
        let mut page: Option<(String, String, String)> = None;
        for line in FUNCTION_PAGE_SOURCE.lines() {
            if let Some(metadata) = line
                .strip_prefix("<!-- GTE_FUNCTION_PAGE: ")
                .and_then(|line| line.strip_suffix(" -->"))
            {
                let (slug, title) = metadata.split_once('|').unwrap_or((metadata, metadata));
                page = Some((slug.to_owned(), title.to_owned(), String::new()));
            } else if line == "<!-- END_GTE_FUNCTION_PAGE -->" {
                if let Some((slug, title, source)) = page.take() {
                    chapters.push(BookChapter {
                        slug,
                        title: format!("Function: {title}"),
                        source: source.trim().to_owned(),
                    });
                }
            } else if let Some((_, _, source)) = page.as_mut() {
                source.push_str(line);
                source.push('\n');
            }
        }
        chapters
    })
}
```

Where it fits

The implementation lives in the focused `web::book::catalog` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `book_chapter_index`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 068 of 111: `web::book::catalog::book_chapter_index`

Source: `src/web/book/catalog.rs:111`

Responsibility

Finds the Guidebook page index for a URL slug.

Signature and data contract

```
pub(in crate::web) fn book_chapter_index(slug: &str) -> Option<usize>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn book_chapter_index(slug: &str) -> Option<usize> {
    book_chapters()
        .iter()
        .position(|chapter| chapter.slug == slug)
}
```

Where it fits

The implementation lives in the focused `web::book::catalog` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `book_chapters`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 069 of 111: `web::book::inline::inline_markdown`

Source: `src/web/book/inline.rs:4`

Responsibility

Escapes text and recognizes only inline code, strong emphasis, and validated links.

Signature and data contract

```
pub(super) fn inline_markdown(text: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn inline_markdown(text: &str) -> String {
    let mut output = String::new();
    let mut index = 0;
    while index < text.len() {
        let tail = &text[index..];
        if let Some(code) = tail.strip_prefix('`') {
            if let Some(end) = code.find('`') {
                output.push_str(&format!("<code>{}</code>", html_escape(&code[..end])));
                index += end + 2;
                continue;
            }
        }
        if let Some(strong) = tail.strip_prefix("**") {
            if let Some(end) = strong.find("**") {
                output.push_str(&format!(
                    "<strong>{}</strong>",
                    inline_markdown(&strong[..end])
                ));
                index += end + 4;
                continue;
            }
        }
        if let Some(link) = tail.strip_prefix('[') {
            if let Some(label_end) = link.find("](") {
                let target_start = label_end + 2;
                if let Some(target_end) = link[target_start..].find(')') {
                    let label = &link[..label_end];
                    let target = &link[target_start..target_start + target_end];
                    if let Some(href) = safe_markdown_href(target) {
                        output.push_str(&format!(
                            "<a href=\"{}\">{}</a>",
                            html_escape(&href),
                            inline_markdown(label)
                        ));
                    } else {
                        output.push_str(&inline_markdown(label));
                    }
                    index += target_start + target_end + 2;
                    continue;
                }
            }
        }
        let character = tail.chars().next().expect("index is before the end");
        output.push_str(&html_escape(&character.to_string()));
        index += character.len_utf8();
    }
    output
}
```

Where it fits

The implementation lives in the focused `web::book::inline` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `safe_markdown_href`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 070 of 111: `web::book::inline::safe_markdown_href`

Source: `src/web/book/inline.rs:53`

Responsibility

Allows a small set of safe external and local links and rewrites reference.md anchors.

Signature and data contract

```
fn safe_markdown_href(target: &str) -> Option<String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn safe_markdown_href(target: &str) -> Option<String> {
    if target.starts_with("https://")
        || target.starts_with("http://")
        || target.starts_with("/guide")
        || target.starts_with('#')
    {
        Some(target.to_owned())
    } else if let Some(fragment) = target.strip_prefix("reference.md") {
        Some(format!("/guide/reference{fragment}"))
    } else {
        None
    }
}
```

Where it fits

The implementation lives in the focused `web::book::inline` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `inline_markdown`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 071 of 111: `web::book::markdown::render_markdown`

Source: `src/web/book/markdown.rs:8`

Responsibility

Converts the documented Markdown subset into HTML while tracking paragraphs, lists, and fenced code.

Signature and data contract

```
pub(super) fn render_markdown(source: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn render_markdown(source: &str) -> String {
    let mut output = String::new();
    let mut paragraph = Vec::new();
    let mut list_kind: Option<bool> = None;
    let mut code_language: Option<String> = None;

    for line in source.lines() {
        if code_language.is_some() {
            if line.starts_with("```") {
                output.push_str("</code></pre>");
                code_language = None;
            } else {
                output.push_str(&html_escape(line));
                output.push('\n');
            }
            continue;
        }
        if line.starts_with("```") {
            flush_markdown_paragraph(&mut paragraph, &mut output);
            close_markdown_list(&mut list_kind, &mut output);
            let language = line.trim_start_matches('`').trim();
            let language_class = if language.is_empty() {
                String::new()
            } else {
                format!(" class=\"language-{}\"", html_escape(language))
            };
            output.push_str(&format!("<pre><code{language_class}>"));
            code_language = Some(language.to_owned());
            continue;
        }
        if line.trim().is_empty() {
            flush_markdown_paragraph(&mut paragraph, &mut output);
            close_markdown_list(&mut list_kind, &mut output);
            continue;
        }
        if let Some((level, heading)) = markdown_heading(line) {
            flush_markdown_paragraph(&mut paragraph, &mut output);
            close_markdown_list(&mut list_kind, &mut output);
            let id = heading_slug(heading);
            output.push_str(&format!(
                "<h{level} id=\"{id}\">{}</h{level}>",
                inline_markdown(heading)
            ));
            continue;
        }
        if line.trim() == "---" || line.trim() == "***" {
            flush_markdown_paragraph(&mut paragraph, &mut output);
            close_markdown_list(&mut list_kind, &mut output);
            output.push_str("<hr>");
            continue;
        }
        if let Some(quote) = line.strip_prefix("> ") {
```

```
            flush_markdown_paragraph(&mut paragraph, &mut output);
            close_markdown_list(&mut list_kind, &mut output);
            output.push_str(&format!(
                "<blockquote><p>{}</p></blockquote>",
                inline_markdown(quote)
            ));
            continue;
        }
        if let Some(item) = line.strip_prefix("- ").or_else(|| line.strip_prefix("* ")) {
            flush_markdown_paragraph(&mut paragraph, &mut output);
            open_markdown_list(false, &mut list_kind, &mut output);
            output.push_str(&format!("<li>{}</li>", inline_markdown(item)));
            continue;
        }
        if let Some((_, item)) = line.split_once(". ").filter(|(number, _)| {
            !number.is_empty() && number.chars().all(|character| character.is_ascii_digit())
        }) {
            flush_markdown_paragraph(&mut paragraph, &mut output);
            open_markdown_list(true, &mut list_kind, &mut output);
            output.push_str(&format!("<li>{}</li>", inline_markdown(item)));
            continue;
        }
        close_markdown_list(&mut list_kind, &mut output);
        paragraph.push(line.trim().to_owned());
    }
    flush_markdown_paragraph(&mut paragraph, &mut output);
    close_markdown_list(&mut list_kind, &mut output);
    if code_language.is_some() {
        output.push_str("</code></pre>");
    }
    output
}
```

Where it fits

The implementation lives in the focused `web::book::markdown` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `flush_markdown_paragraph`, `open_markdown_list`, `close_markdown_list`, `markdown_heading`, `heading_slug`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 072 of 111: `web::book::markdown::flush_markdown_paragraph`

Source: `src/web/book/markdown.rs:93`

Responsibility

Joins a pending paragraph's wrapped source lines and emits one escaped HTML paragraph.

Signature and data contract

```
fn flush_markdown_paragraph(paragraph: &mut Vec<String>, output: &mut String)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn flush_markdown_paragraph(paragraph: &mut Vec<String>, output: &mut String) {
    if paragraph.is_empty() {
        return;
    }
    let text = paragraph.join(" ");
    output.push_str(&format!("<p>{}</p>", inline_markdown(&text)));
    paragraph.clear();
}
```

Where it fits

The implementation lives in the focused `web::book::markdown` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_markdown`, `open_markdown_list`, `close_markdown_list`, `markdown_heading`, `heading_slug`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 073 of 111: `web::book::markdown::open_markdown_list`

Source: `src/web/book/markdown.rs:102`

Responsibility

Opens an ordered or unordered list, closing a prior list if its kind changed.

Signature and data contract

```
fn open_markdown_list(ordered: bool, current: &mut Option<bool>, output: &mut String)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn open_markdown_list(ordered: bool, current: &mut Option<bool>, output: &mut String) {
    if *current == Some(ordered) {
        return;
    }
    close_markdown_list(current, output);
    output.push_str(if ordered { "<ol>" } else { "<ul>" });
    *current = Some(ordered);
}
```

Where it fits

The implementation lives in the focused `web::book::markdown` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_markdown`, `flush_markdown_paragraph`, `close_markdown_list`, `markdown_heading`, `heading_slug`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 074 of 111: `web::book::markdown::close_markdown_list`

Source: `src/web/book/markdown.rs:111`

Responsibility

Closes the currently open Markdown list, if there is one.

Signature and data contract

```
fn close_markdown_list(current: &mut Option<bool>, output: &mut String)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn close_markdown_list(current: &mut Option<bool>, output: &mut String) {
    match current.take() {
        Some(true) => output.push_str("</ol>"),
        Some(false) => output.push_str("</ul>"),
        None => {}
    }
}
```

Where it fits

The implementation lives in the focused `web::book::markdown` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_markdown`, `flush_markdown_paragraph`, `open_markdown_list`, `markdown_heading`, `heading_slug`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 075 of 111: `web::book::markdown::markdown_heading`

Source: `src/web/book/markdown.rs:119`

Responsibility

Recognizes one through six leading Markdown heading markers followed by a space.

Signature and data contract

```
fn markdown_heading(line: &str) -> Option<(usize, &str)>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn markdown_heading(line: &str) -> Option<(usize, &str)> {
    let hashes = line
        .chars()
        .take_while(|character| *character == '#')
        .count();
    (1..=6)
        .contains(&hashes)
        .then(|| line.get(hashes..).unwrap_or_default())
        .and_then(|heading| heading.strip_prefix(' ').map(|text| (hashes, text)))
}
```

Where it fits

The implementation lives in the focused `web::book::markdown` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_markdown`, `flush_markdown_paragraph`, `open_markdown_list`, `close_markdown_list`, `heading_slug`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 076 of 111: `web::book::markdown::heading_slug`

Source: `src/web/book/markdown.rs:130`

Responsibility

Builds a stable fragment id from lowercase alphanumeric heading text.

Signature and data contract

```
fn heading_slug(heading: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn heading_slug(heading: &str) -> String {
    let mut slug = String::new();
    let mut separator = false;
    for character in heading
        .chars()
        .flat_map(|character| character.to_lowercase())
    {
        if character.is_ascii_alphanumeric() {
            if separator && !slug.is_empty() {
                slug.push('-');
            }
            slug.push(character);
            separator = false;
        } else if character.is_whitespace() || character == '-' {
            separator = true;
        }
    }
    slug
}
```

Where it fits

The implementation lives in the focused `web::book::markdown` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_markdown`, `flush_markdown_paragraph`, `open_markdown_list`, `close_markdown_list`, `markdown_heading`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 077 of 111: `web::book::reader::render_book`

Source: `src/web/book/reader.rs:8`

Responsibility

Builds the Guidebook sidebar, active reading page, breadcrumb, and previous/next navigation.

Signature and data contract

```
pub(in crate::web) fn render_book(slug: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn render_book(slug: &str) -> String {
    let Some(index) = book_chapter_index(slug) else {
        return String::new();
    };
    let chapters = book_chapters();
    let chapter = &chapters[index];
    let mut sidebar = String::new();
    for (chapter_index, item) in chapters.iter().enumerate() {
        let active = if chapter_index == index {
            " class=\"active\" aria-current=\"page\""
        } else {
            ""
        };
        sidebar.push_str(&format!(
            "<a{active} href=\"/guide/{}\"><span class=\"chapter-number\">{:02}</span>{}</a>",
            item.slug,
            chapter_index + 1,
            html_escape(&item.title)
        ));
    }
    let previous = index.checked_sub(1).map(|previous_index| {
        let item = &chapters[previous_index];
        format!(
            "<a class=\"chapter-card previous\" href=\"/guide/{}\"><small>PREVIOUS</small><span>← {}</span></a>",
            item.slug,
            html_escape(&item.title)
        )
    }).unwrap_or_else(|| "<span></span>".to_owned());
    let next = chapters.get(index + 1).map(|item| {
        format!(
            "<a class=\"chapter-card next\" href=\"/guide/{}\"><small>NEXT</small><span>{} →</span></a>",
            item.slug,
            html_escape(&item.title)
        )
    }).unwrap_or_default();
    let body = format!(
        include_str!("../../../assets/templates/book.html"),
        html_escape(&chapter.title),
        html_escape(&chapter.title),
        render_markdown(&chapter.source),
        previous,
        next,
        sidebar = sidebar,
    );
    format!(
        include_str!("../../../assets/templates/book-page.html"),
        html_escape(&chapter.title),
        BOOK_STYLE = BOOK_STYLE,
        body = body,
    )
}
```

Where it fits

The implementation lives in the focused `web::book::reader` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `download_book_markdown`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 078 of 111: `web::book::reader::download_book_markdown`

Source: `src/web/book/reader.rs:60`

Responsibility

Concatenates the embedded Guidebook pages into the downloadable Markdown edition.

Signature and data contract

```
pub(in crate::web) fn download_book_markdown() -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn download_book_markdown() -> String {
    let mut markdown =
        String::from("# The GTE Book\n\nA Rust chess app and terminal learning project.\n");
    for chapter in book_chapters() {
        markdown.push_str("\n\n---\n\n");
        markdown.push_str(chapter.source.trim());
        markdown.push('\n');
    }
    markdown
}
```

Where it fits

The implementation lives in the focused `web::book::reader` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_book`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 079 of 111: `web::handlers::guesses::submit_elo_guess`

Source: `src/web/handlers/guesses.rs:8`

Responsibility

Scores only the requested player side or sides and marks the session's ratings revealed.

Signature and data contract

```
pub(in crate::web) fn submit_elo_guess(params: &HashMap<String, String>) -> HttpResponse
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn submit_elo_guess(params: &HashMap<String, String>) -> HttpResponse {
    let Some(id) = params.get("sid") else {
        return HttpResponse::Page(render_home(
            "This game session has expired. Start a new round.",
        ));
    };
    let mut store = match sessions().lock() {
        Ok(store) => store,
        Err(_) => {
            return HttpResponse::Page(render_home("The local game session could not be read."));
        }
    };
    let Some(session) = store.get_mut(id) else {
        return HttpResponse::Page(render_home(
            "This game session has expired. Start a new round.",
        ));
    };
    if session.mode != GameMode::Elo {
        return HttpResponse::Redirect(format!("/play?sid={}", html_escape(id)));
    }

    let mut lines = Vec::new();
    if session.target != "black" {
        score_one(
            &mut lines,
            "White",
            params.get("white").map(String::as_str).unwrap_or_default(),
            session.game.white_rating,
            &session.difficulty,
        );
    }
    if session.target != "white" {
        score_one(
            &mut lines,
            "Black",
            params.get("black").map(String::as_str).unwrap_or_default(),
            session.game.black_rating,
            &session.difficulty,
        );
    }
    session.feedback = lines.join("<br>");
    session.revealed = true;
    HttpResponse::Redirect(format!("/play?sid={}", html_escape(id)))
}
```

Where it fits

The implementation lives in the focused `web::handlers::guesses` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `score_one`, `submit_opening_guess`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 080 of 111: `web::handlers::guesses::score_one`

Source: `src/web/handlers/guesses.rs:53`

Responsibility

Validates one numeric guess, handles missing ratings, computes error, and appends tolerance feedback.

Signature and data contract

```
fn score_one(
    lines: &mut Vec<String>,
    color: &str,
    raw_guess: &str,
    actual: Option<i32>,
    difficulty: &str,
)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn score_one(
    lines: &mut Vec<String>,
    color: &str,
    raw_guess: &str,
    actual: Option<i32>,
    difficulty: &str,
) {
    let Some(guess) = raw_guess
        .trim()
        .parse::<i32>()
        .ok()
        .filter(|value| (0..=4000).contains(value))
    else {
        lines.push(format!("{color}: enter a numeric rating guess."));
        return;
    };
    let Some(actual) = actual else {
        lines.push(format!(
            "{color}: no rating was included with this game record."
        ));
        return;
    };
    let error = (guess - actual).abs();
    let outcome = if rating_is_close(guess, actual, difficulty) {
        "inside tolerance"
    } else {
        "outside tolerance"
    };
    lines.push(format!(
        "{color}: you guessed {guess}; actual rating {actual}; off by {error} · {outcome}."
    ));
}
```

Where it fits

The implementation lives in the focused `web::handlers::guesses` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `submit_elo_guess`, `submit_opening_guess`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 081 of 111: `web::handlers::guesses::submit_opening_guess`

Source: `src/web/handlers/guesses.rs:86`

Responsibility

Validates the active mode, normalizes the answer through chess.rs, and stores the revealed result.

Signature and data contract

```
pub(in crate::web) fn submit_opening_guess(params: &HashMap<String, String>) -> HttpResponse
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn submit_opening_guess(params: &HashMap<String, String>) -> HttpResponse {
    let Some(id) = params.get("sid") else {
        return HttpResponse::Page(render_home(
            "This game session has expired. Start a new round.",
        ));
    };
    let mut store = match sessions().lock() {
        Ok(store) => store,
        Err(_) => {
            return HttpResponse::Page(render_home("The local game session could not be read."));
        }
    };
    let Some(session) = store.get_mut(id) else {
        return HttpResponse::Page(render_home(
            "This game session has expired. Start a new round.",
        ));
    };
    if session.mode != GameMode::Opening {
        return HttpResponse::Redirect(format!("/play?sid={}", html_escape(id)));
    }
    let guess = params.get("guess").map(String::as_str).unwrap_or_default();
    session.feedback = if opening_matches(guess, &session.game.opening) {
        format!(
            "Correct — the opening is {}.",
            html_escape(&session.game.opening)
        )
    } else if guess.trim().is_empty() {
        "Enter an opening name before submitting.".to_owned()
    } else {
        format!(
            "Not quite. The opening is {}.",
            html_escape(&session.game.opening)
        )
    };
    session.revealed = true;
    HttpResponse::Redirect(format!("/play?sid={}", html_escape(id)))
}
```

Where it fits

The implementation lives in the focused `web::handlers::guesses` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `submit_elo_guess`, `score_one`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 082 of 111: `web::handlers::navigation::render_session`

Source: `src/web/handlers/navigation.rs:7`

Responsibility

Looks up a session id and renders the saved game or explains that the session is unavailable.

Signature and data contract

```
pub(in crate::web) fn render_session(params: &HashMap<String, String>) -> HttpResponse
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn render_session(params: &HashMap<String, String>) -> HttpResponse {
    let Some(id) = params.get("sid") else {
        return HttpResponse::Page(render_home(
            "This game session has expired. Start a new round.",
        ));
    };
    match sessions().lock() {
        Ok(store) => match store.get(id) {
            Some(session) => HttpResponse::Page(render_game(session, id)),
            None => HttpResponse::Page(render_home(
                "This game session has expired. Start a new round.",
            )),
        },
        Err(_) => HttpResponse::Page(render_home(
            "The local game session could not be read. Start a new round.",
        )),
    }
}
```

Where it fits

The implementation lives in the focused `web::handlers::navigation` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `update_move`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 083 of 111: `web::handlers::navigation::update_move`

Source: `src/web/handlers/navigation.rs:26`

Responsibility

Parses and clamps the requested ply, clears stale feedback, then redirects to the session page.

Signature and data contract

```
pub(in crate::web) fn update_move(params: &HashMap<String, String>) -> HttpResponse
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn update_move(params: &HashMap<String, String>) -> HttpResponse {
    let Some(id) = params.get("sid") else {
        return HttpResponse::Page(render_home(
            "This game session has expired. Start a new round.",
        ));
    };
    if let Ok(mut store) = sessions().lock() {
        if let Some(session) = store.get_mut(id) {
            let requested = params
                .get("ply")
                .and_then(|value| value.parse::<usize>().ok())
                .unwrap_or(session.ply);
            session.ply = requested.min(max_ply(session));
            session.feedback.clear();
        }
    }
    HttpResponse::Redirect(format!("/play?sid={}", html_escape(id)))
}
```

Where it fits

The implementation lives in the focused `web::handlers::navigation` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_session`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 084 of 111: `web::handlers::setup::load_game`

Source: `src/web/handlers/setup.rs:10`

Responsibility

Normalizes the round settings, selects demo/archive/link data, and keeps a user-readable API fallback notice.

Signature and data contract

```
pub(in crate::web) fn load_game(params: &HashMap<String, String>) -> HttpResponse
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn load_game(params: &HashMap<String, String>) -> HttpResponse {
    let mode = if params.get("mode").map(String::as_str) == Some("opening") {
        GameMode::Opening
    } else {
        GameMode::Elo
    };
    let difficulty = params
        .get("difficulty")
        .map(|value| normalize_difficulty(value))
        .unwrap_or_else(|| "medium".to_owned());
    let target = params
        .get("target")
        .map(|value| normalize_target(value))
        .unwrap_or_else(|| "both".to_owned());
    let username = params.get("username").map(String::as_str).unwrap_or("erik");
    let source = params.get("source").map(String::as_str).unwrap_or("demo");

    let (game, note) = match source {
        "random" => match api::random_public_game(username) {
            Ok(game) => (game, String::new()),
            Err(error) => (
                ChessGame::demo(),
                format!("API fallback: {error} The included sample game is shown instead."),
            ),
        },
        "link" => {
            let url = params.get("url").map(String::as_str).unwrap_or_default();
            match api::game_from_archive_link(username, url) {
                Ok(game) => (game, String::new()),
                Err(error) => (
                    ChessGame::demo(),
                    format!(
                        "Game link unavailable: {error} Demo data is shown so you can still explore the board."
                    ),
                ),
            }
        }
        _ => (
            ChessGame::demo(),
            "Demo data · this built-in game works without a network connection.".to_owned(),
        ),
    };
    if game.ply_count() == 0 {
        let demo = ChessGame::demo();
        let note = if note.is_empty() {
            "The loaded PGN had no readable moves, so the demo game is shown.".to_owned()
        } else {
            note
        };
        return create_session(demo, mode, difficulty, target, note);
    }
    create_session(game, mode, difficulty, target, note)
```

```
}
```

Where it fits

The implementation lives in the focused `web::handlers::setup` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `create_session`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 085 of 111: `web::handlers::setup::create_session`

Source: `src/web/handlers/setup.rs:64`

Responsibility

Allocates a session id, chooses the starting ply for the mode, stores state, and redirects to play.

Signature and data contract

```
fn create_session(
    game: ChessGame,
    mode: GameMode,
    difficulty: String,
    target: String,
    note: String,
) -> HttpResponse
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn create_session(
    game: ChessGame,
    mode: GameMode,
    difficulty: String,
    target: String,
    note: String,
) -> HttpResponse {
    let id = NEXT_SESSION.fetch_add(1, Ordering::Relaxed).to_string();
    let ply = if mode == GameMode::Opening {
        8.min(game.ply_count())
    } else {
        0
    };
    let session = Session {
        game,
        mode,
        difficulty,
        target,
        ply,
        note,
        feedback: String::new(),
        revealed: false,
    };
    if let Ok(mut store) = sessions().lock() {
        store.insert(id.clone(), session);
    }
    HttpResponse::Redirect(format!("/play?sid={id}"))
}
```

Where it fits

The implementation lives in the focused `web::handlers::setup` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `load_game`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 086 of 111: `web::http::serve`

Source: `src/web/http.rs:17`

Responsibility

Binds the localhost HTTP listener, prints the local URLs, and serially handles connections.

Signature and data contract

```
pub fn serve()
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub fn serve() {
    let listener = match TcpListener::bind("127.0.0.1:7878") {
        Ok(listener) => listener,
        Err(error) => {
            eprintln!("Could not start the ChessProject web app: {error}");
            return;
        }
    };
    println!("ChessProject is running at http://127.0.0.1:7878");
    println!("Guidebook: http://127.0.0.1:7878/guide");
    println!("Press Ctrl+C to stop the local server.");

    for incoming in listener.incoming() {
        match incoming {
            Ok(stream) => handle_connection(stream),
            Err(error) => eprintln!("Browser connection failed: {error}"),
        }
    }
}
```

Where it fits

The implementation lives in the focused `web::http` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `handle_connection`, `write_response`, `write_download`, `write_pdf`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 087 of 111: `web::http::handle_connection`

Source: `src/web/http.rs:37`

Responsibility

Reads one request line, permits GET, dispatches a route, and writes the matching HTTP response.

Signature and data contract

```
fn handle_connection(mut stream: TcpStream)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn handle_connection(mut stream: TcpStream) {
    let _ = stream.set_read_timeout(Some(std::time::Duration::from_secs(3)));
    let mut buffer = [0u8; 16_384];
    let count = match stream.read(&mut buffer) {
        Ok(count) if count > 0 => count,
        _ => return,
    };
    let request = String::from_utf8_lossy(&buffer[..count]);
    let first_line = request.lines().next().unwrap_or_default();
    let mut parts = first_line.split_whitespace();
    let method = parts.next().unwrap_or("GET");
    let target = parts.next().unwrap_or("/");
    if method != "GET" {
        write_response(
            &mut stream,
            405,
            "text/plain; charset=utf-8",
            "Use a browser GET form for this prototype.",
        );
        return;
    }
    let response = route(target);
    match response {
        HttpResponse::Page(body) => {
            write_response(&mut stream, 200, "text/html; charset=utf-8", &body)
        }
        HttpResponse::Download(body) => write_download(&mut stream, "GTE-Guidebook.md", &body),
        HttpResponse::Pdf(body) => write_pdf(&mut stream, &body),
        HttpResponse::Redirect(location) => {
            let head = format!(
                "HTTP/1.1 303 See Other\r\nLocation: {location}\r\nContent-Length: 0\r\nConnection: close\r\n\r\n"
            );
            let _ = stream.write_all(head.as_bytes());
        }
        HttpResponse::NotFound => write_response(
            &mut stream,
            404,
            "text/plain; charset=utf-8",
            "Page not found.",
        ),
    }
}
```

Where it fits

The implementation lives in the focused `web::http` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `serve`, `write_response`, `write_download`, `write_pdf`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 088 of 111: `web::http::write_response`

Source: `src/web/http.rs:80`

Responsibility

Writes a standard HTTP response header and its body to the client socket.

Signature and data contract

```
fn write_response<W: Write>(stream: &mut W, status: u16, content_type: &str, body: &str)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn write_response<W: Write>(stream: &mut W, status: u16, content_type: &str, body: &str) {
    let reason = match status {
        200 => "OK",
        404 => "Not Found",
        405 => "Method Not Allowed",
        _ => "Error",
    };
    let head = format!(
        "HTTP/1.1 {status} {reason}\r\nContent-Type: {content_type}\r\nContent-Length: {}\r\nConnection:
          close\r\nX-Content-Type-Options: nosniff\r\n\r\n",
        body.len()
    );
    let _ = stream.write_all(head.as_bytes());
    let _ = stream.write_all(body.as_bytes());
}
```

Where it fits

The implementation lives in the focused `web::http` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `serve`, `handle_connection`, `write_download`, `write_pdf`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 089 of 111: `web::http::write_download`

Source: `src/web/http.rs:95`

Responsibility

Sends Markdown with an attachment disposition and the selected filename.

Signature and data contract

```
fn write_download<W: Write>(stream: &mut W, filename: &str, body: &str)
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn write_download<W: Write>(stream: &mut W, filename: &str, body: &str) {
    let head = format!(
        "HTTP/1.1 200 OK\r\nContent-Type: text/markdown; charset=utf-8\r\nContent-Disposition: attachment;
          filename=\"{filename}\"\r\nContent-Length: {}\r\nConnection: close\r\nX-Content-Type-Options:
          nosniff\r\n\r\n",
        body.len()
    );
    let _ = stream.write_all(head.as_bytes());
    let _ = stream.write_all(body.as_bytes());
}
```

Where it fits

The implementation lives in the focused `web::http` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `serve`, `handle_connection`, `write_response`, `write_pdf`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 090 of 111: `web::http::write_pdf`

Source: `src/web/http.rs:104`

Responsibility

Sends the embedded function appendix bytes with an application/pdf content type and download filename.

Signature and data contract

```
fn write_pdf<W: Write>(stream: &mut W, body: &[u8])
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn write_pdf<W: Write>(stream: &mut W, body: &[u8]) {
    let head = format!(
        "HTTP/1.1 200 OK\r\nContent-Type: application/pdf\r\nContent-Disposition: attachment;
          filename=\"GTE-Function-Appendix.pdf\"\r\nContent-Length: {}\r\nConnection: close\r\nX-Content-Type-Options:
           nosniff\r\n\r\n",
        body.len()
    );
    let _ = stream.write_all(head.as_bytes());
    let _ = stream.write_all(body);
}
```

Where it fits

The implementation lives in the focused `web::http` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `serve`, `handle_connection`, `write_response`, `write_download`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 091 of 111: `web::render::board::render_board`

Source: `src/web/render/board.rs:4`

Responsibility

Converts all 64 board cells into labeled squares for the accessible full board.

Signature and data contract

```
pub(super) fn render_board(position: &[char; 64]) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn render_board(position: &[char; 64]) -> String {
    let mut output =
        String::from("<div class=\"board\" role=\"img\" aria-label=\"Chess position\">");
    for row in 0..8 {
        for column in 0..8 {
            let piece = position[row * 8 + column];
            let square = if (row + column) % 2 == 0 {
                "light-square"
            } else {
                "dark-square"
            };
            let symbol = chess_symbol(piece);
            let label = if piece == '.' {
                "empty".to_owned()
            } else {
                format!(
                    "{} on {}{}",
                    piece_name(piece),
                    (b'a' + column as u8) as char,
                    8 - row
                )
            };
            output.push_str(&format!(
                "<div class=\"square {square}\" aria-label=\"{}\">{symbol}</div>",
                html_escape(&label)
            ));
        }
    }
    output.push_str("</div>");
    output
}
```

Where it fits

The implementation lives in the focused `web::render::board` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `mini_board`, `chess_symbol`, `piece_name`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 092 of 111: `web::render::board::mini_board`

Source: `src/web/render/board.rs:36`

Responsibility

Renders the compact decorative board used on the home page.

Signature and data contract

```
pub(super) fn mini_board(position: &[char; 64]) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn mini_board(position: &[char; 64]) -> String {
    let mut output = String::from("<div class=\"mini-board-grid\">");
    for row in 0..8 {
        for column in 0..8 {
            let piece = position[row * 8 + column];
            let square = if (row + column) % 2 == 0 {
                "light-square"
            } else {
                "dark-square"
            };
            output.push_str(&format!(
                "<div class=\"square {square}\">{}</div>",
                chess_symbol(piece)
            ));
        }
    }
    output.push_str("</div>");
    output
}
```

Where it fits

The implementation lives in the focused `web::render::board` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_board`, `chess_symbol`, `piece_name`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 093 of 111: `web::render::board::chess_symbol`

Source: `src/web/render/board.rs:56`

Responsibility

Maps internal piece letters to Unicode chess glyphs, returning an empty string for empty squares.

Signature and data contract

```
fn chess_symbol(piece: char) -> &'static str
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn chess_symbol(piece: char) -> &'static str {
    match piece {
        'K' => "♔",
        'Q' => "♕",
        'R' => "♖",
        'B' => "♗",
        'N' => "♘",
        'P' => "♙",
        'k' => "♚",
        'q' => "♛",
        'r' => "♜",
        'b' => "♝",
        'n' => "♞",
        'p' => "♟",
        _ => "",
    }
}
```

Where it fits

The implementation lives in the focused `web::render::board` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_board`, `mini_board`, `piece_name`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 094 of 111: `web::render::board::piece_name`

Source: `src/web/render/board.rs:74`

Responsibility

Maps an internal piece letter to a spoken accessibility label.

Signature and data contract

```
fn piece_name(piece: char) -> &'static str
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn piece_name(piece: char) -> &'static str {
    match piece.to_ascii_uppercase() {
        'K' => "king",
        'Q' => "queen",
        'R' => "rook",
        'B' => "bishop",
        'N' => "knight",
        _ => "pawn",
    }
}
```

Where it fits

The implementation lives in the focused `web::render::board` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_board`, `mini_board`, `chess_symbol`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 095 of 111: `web::render::game::render_game`

Source: `src/web/render/game.rs:8`

Responsibility

Composes the board, player rows, move list, metadata, navigation, and mode-specific answer panel.

Signature and data contract

```
pub(in crate::web) fn render_game(session: &Session, id: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn render_game(session: &Session, id: &str) -> String {
    let game = &session.game;
    let mode_label = if session.mode == GameMode::Elo {
        "GUESS THE ELO"
    } else {
        "GUESS THE OPENING"
    };
    let limit = max_ply(session);
    let move_list = render_move_list(game, id, session.ply, limit);
    let board = render_board(game.position_at(session.ply));
    let note = if session.note.is_empty() {
        String::new()
    } else {
        format!("<div class=\"toast\">{}</div>", html_escape(&session.note))
    };
    let game_link = if game.source_url.starts_with("https://") {
        format!(
            "<a class=\"source-link\" href=\"{}\" target=\"_blank\" rel=\"noreferrer\">Open on Chess.com ↗</a>",
            html_escape(&game.source_url)
        )
    } else {
        String::new()
    };
    let feedback = if session.feedback.is_empty() {
        String::new()
    } else {
        format!("<div class=\"result-box\">{}</div>", session.feedback)
    };
    let sidebar = if session.mode == GameMode::Elo {
        render_elo_panel(session, id, &feedback)
    } else {
        render_opening_panel(session, id, &feedback)
    };
    let title = if session.mode == GameMode::Elo {
        "Guess the Elo"
    } else {
        "Guess the Opening"
    };
    let body = format!(
        include_str!("../../../assets/templates/game.html"),
        if game.demo {
            "DEMO DATA"
        } else {
            "CHESS.COM ARCHIVE"
        },
        html_escape(title),
        render_player(
            "black",
            &game.black_name,
            game.black_rating,
            session.revealed && session.target != "white"
        ),
```

```
        board,
        render_player(
            "white",
            &game.white_name,
            game.white_rating,
            session.revealed && session.target != "black"
        ),
        session.ply.saturating_sub(1),
        session.ply,
        limit,
        session.ply.saturating_add(1).min(limit),
        limit,
        limit,
        move_list,
        sidebar,
        html_escape(&game.result),
        html_escape(&game.time_control),
        html_escape(&game.date),
        if game.eco.is_empty() {
            "—".to_owned()
        } else {
            html_escape(&game.eco)
        },
        game_link,
        mode_label = mode_label,
        id = id,
    );
    page(title, &note, &body)
}
```

Where it fits

The implementation lives in the focused `web::render::game` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_player`, `render_move_list`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 096 of 111: `web::render::game::render_player`

Source: `src/web/render/game.rs:90`

Responsibility

Renders a player row while hiding its rating until the round's reveal state permits it.

Signature and data contract

```
fn render_player(color: &str, name: &str, rating: Option<i32>, revealed: bool) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn render_player(color: &str, name: &str, rating: Option<i32>, revealed: bool) -> String {
    let (icon, class, label) = if color == "white" {
        ("♙", "white-dot", "White")
    } else {
        ("▲", "black-dot", "Black")
    };
    let rating_text = if revealed {
        rating
            .map(|value| format!("Rating · {value}"))
            .unwrap_or_else(|| "Rating unavailable".to_owned())
    } else {
        "Rating hidden".to_owned()
    };
    let visible_rating = if revealed {
        String::new()
    } else {
        "•••".to_owned()
    };
    format!(
        "<div class=\"player-line\"><span class=\"piece-dot
          {class}\">{icon}</span><div><strong>{}</strong><small>{label} · {}</small></div><span
          class=\"rating-lock\">{visible_rating}</span></div>",
        html_escape(name),
        html_escape(&rating_text)
    )
}
```

Where it fits

The implementation lives in the focused `web::render::game` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_game`, `render_move_list`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 097 of 111: `web::render::game::render_move_list`

Source: `src/web/render/game.rs:115`

Responsibility

Turns each visible SAN token into a navigation link and marks the selected ply.

Signature and data contract

```
fn render_move_list(
    game: &ChessGame,
    id: &str,
    selected_ply: usize,
    visible_plies: usize,
) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn render_move_list(
    game: &ChessGame,
    id: &str,
    selected_ply: usize,
    visible_plies: usize,
) -> String {
    if game.moves.is_empty() {
        return "<p class=\"muted\">No readable moves in this PGN.</p>".to_owned();
    }
    let mut output = String::from("<div class=\"move-grid\">");
    for (index, san) in game.moves.iter().take(visible_plies).enumerate() {
        let ply = index + 1;
        let move_number = (index / 2) + 1;
        let class = if ply == selected_ply {
            "move-token selected"
        } else {
            "move-token"
        };
        if index % 2 == 0 {
            output.push_str(&format!(
                "<span class=\"move-number\">{move_number}.</span>"
            ));
        }
        output.push_str(&format!(
            "<a class=\"{class}\" href=\"/move?sid={id}&amp;ply={ply}\">{}</a>",
            html_escape(san)
        ));
    }
    output.push_str("</div>");
    output
}
```

Where it fits

The implementation lives in the focused `web::render::game` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_game`, `render_player`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 098 of 111: `web::render::home::render_home`

Source: `src/web/render/home.rs:6`

Responsibility

Builds the overview, GothamChess link, formal description, challenge forms, and demo links.

Signature and data contract

```
pub(in crate::web) fn render_home(message: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn render_home(message: &str) -> String {
    let body = format!(
        include_str!("../../../assets/templates/home.html"),
        mini_board(ChessGame::demo().position_at(8)),
        mini_board(ChessGame::demo().position_at(8))
    );
    let notice = if message.is_empty() {
        String::new()
    } else {
        format!("<div class=\"toast\">{}</div>", html_escape(message))
    };
    page("GTE · Read the Game", &notice, &body)
}
```

Where it fits

The implementation lives in the focused `web::render::home` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 099 of 111: `web::render::html::page`

Source: `src/web/render/html.rs:4`

Responsibility

Wraps a rendered app body in the shared HTML shell and app stylesheet.

Signature and data contract

```
pub(in crate::web) fn page(title: &str, notice: &str, body: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn page(title: &str, notice: &str, body: &str) -> String {
    format!(
        include_str!("../../../assets/templates/page.html"),
        html_escape(title),
        STYLE,
        notice,
        body
    )
}
```

Where it fits

The implementation lives in the focused `web::render::html` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `html_escape`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 100 of 111: `web::render::html::html_escape`

Source: `src/web/render/html.rs:14`

Responsibility

Replaces five HTML-sensitive characters before untrusted text enters markup.

Signature and data contract

```
pub(in crate::web) fn html_escape(text: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(in crate::web) fn html_escape(text: &str) -> String {
    text.replace('&', "&amp;")
        .replace('<', "&lt;")
        .replace('>', "&gt;")
        .replace('"', "&quot;")
        .replace('\'', "&#39;")
}
```

Where it fits

The implementation lives in the focused `web::render::html` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `page`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 101 of 111: `web::render::panels::render_elo_panel`

Source: `src/web/render/panels.rs:6`

Responsibility

Creates the rating form with inputs only for the selected side and the selected tolerance displayed.

Signature and data contract

```
pub(super) fn render_elo_panel(session: &Session, id: &str, feedback: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn render_elo_panel(session: &Session, id: &str, feedback: &str) -> String {
    let white_input = if session.target == "black" {
        String::new()
    } else {
        format!(
            "<label>White · {}<input type=number min=0 max=4000 name=white placeholder=\"Your guess\"
              required></label>",
            html_escape(&session.game.white_name)
        )
    };
    let black_input = if session.target == "white" {
        String::new()
    } else {
        format!(
            "<label>Black · {}<input type=number min=0 max=4000 name=black placeholder=\"Your guess\"
              required></label>",
            html_escape(&session.game.black_name)
        )
    };
    let answer_area = if session.revealed {
        feedback.to_owned()
    } else {
        String::new()
    };
    format!(
        include_str!("../../../assets/templates/elo-panel.html"),
        crate::chess::rating_tolerance(&session.difficulty),
        session.difficulty.to_ascii_uppercase(),
        crate::chess::rating_tolerance(&session.difficulty),
        id,
        white_input,
        black_input,
        answer_area,
        session.target.to_ascii_uppercase(),
        session.difficulty.to_ascii_uppercase()
    )
}
```

Where it fits

The implementation lives in the focused `web::render::panels` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_opening_panel`, `opening_sequence`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 102 of 111: `web::render::panels::render_opening_panel`

Source: `src/web/render/panels.rs:42`

Responsibility

Builds the opening answer form and shows the visible short sequence of SAN moves.

Signature and data contract

```
pub(super) fn render_opening_panel(session: &Session, id: &str, feedback: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn render_opening_panel(session: &Session, id: &str, feedback: &str) -> String {
    let feedback = if session.revealed {
        feedback.to_owned()
    } else {
        String::new()
    };
    format!(
        include_str!("../../../assets/templates/opening-panel.html"),
        session.ply,
        html_escape(&opening_sequence(&session.game)),
        id,
        feedback
    )
}
```

Where it fits

The implementation lives in the focused `web::render::panels` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_elo_panel`, `opening_sequence`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 103 of 111: `web::render::panels::opening_sequence`

Source: `src/web/render/panels.rs:57`

Responsibility

Formats up to eight stored SAN plies into conventional numbered chess notation.

Signature and data contract

```
fn opening_sequence(game: &ChessGame) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn opening_sequence(game: &ChessGame) -> String {
    let mut sequence = String::new();
    for (index, san) in game.moves.iter().take(8).enumerate() {
        if index % 2 == 0 {
            sequence.push_str(&format!("{}.", index / 2 + 1));
        }
        sequence.push(' ');
        sequence.push_str(san);
    }
    sequence.trim().to_owned()
}
```

Where it fits

The implementation lives in the focused `web::render::panels` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `render_elo_panel`, `render_opening_panel`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 104 of 111: `web::request::parse_query`

Source: `src/web/request.rs:4`

Responsibility

Decodes ampersand-separated key/value pairs into the request parameter map.

Signature and data contract

```
pub(super) fn parse_query(query: &str) -> HashMap<String, String>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn parse_query(query: &str) -> HashMap<String, String> {
    query
        .split('&')
        .filter(|part| !part.is_empty())
        .filter_map(|part| {
            let (key, value) = part.split_once('=').unwrap_or((part, ""));
            Some((url_decode(key), url_decode(value)))
        })
        .collect()
}
```

Where it fits

The implementation lives in the focused `web::request` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `url_decode`, `hex_value`, `normalize_difficulty`, `normalize_target`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 105 of 111: `web::request::url_decode`

Source: `src/web/request.rs:15`

Responsibility

Decodes plus signs and valid percent escapes before converting bytes into UTF-8 text.

Signature and data contract

```
fn url_decode(value: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn url_decode(value: &str) -> String {
    let bytes = value.as_bytes();
    let mut decoded = Vec::with_capacity(bytes.len());
    let mut index = 0;
    while index < bytes.len() {
        match bytes[index] {
            b'+' => {
                decoded.push(b' ');
                index += 1;
            }
            b'%' if index + 2 < bytes.len() => {
                let high = hex_value(bytes[index + 1]);
                let low = hex_value(bytes[index + 2]);
                if let (Some(high), Some(low)) = (high, low) {
                    decoded.push(high * 16 + low);
                    index += 3;
                } else {
                    decoded.push(bytes[index]);
                    index += 1;
                }
            }
            byte => {
                decoded.push(byte);
                index += 1;
            }
        }
    }
    String::from_utf8_lossy(&decoded).into_owned()
}
```

Where it fits

The implementation lives in the focused `web::request` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `parse_query`, `hex_value`, `normalize_difficulty`, `normalize_target`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 106 of 111: `web::request::hex_value`

Source: `src/web/request.rs:45`

Responsibility

Converts one hexadecimal ASCII digit into its numeric nibble value.

Signature and data contract

```
fn hex_value(byte: u8) -> Option<u8>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
fn hex_value(byte: u8) -> Option<u8> {
    match byte {
        b'0'..=b'9' => Some(byte - b'0'),
        b'a'..=b'f' => Some(byte - b'a' + 10),
        b'A'..=b'F' => Some(byte - b'A' + 10),
        _ => None,
    }
}
```

Where it fits

The implementation lives in the focused `web::request` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `parse_query`, `url_decode`, `normalize_difficulty`, `normalize_target`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 107 of 111: `web::request::normalize_difficulty`

Source: `src/web/request.rs:54`

Responsibility

Restricts user input to easy, medium, or hard and accepts the brief's harm typo as hard.

Signature and data contract

```
pub(super) fn normalize_difficulty(value: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn normalize_difficulty(value: &str) -> String {
    match value.to_ascii_lowercase().as_str() {
        "easy" => "easy".to_owned(),
        "hard" | "harm" => "hard".to_owned(),
        _ => "medium".to_owned(),
    }
}
```

Where it fits

The implementation lives in the focused `web::request` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `parse_query`, `url_decode`, `hex_value`, `normalize_target`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 108 of 111: `web::request::normalize_target`

Source: `src/web/request.rs:62`

Responsibility

Restricts user input to white, black, or the default both-player choice.

Signature and data contract

```
pub(super) fn normalize_target(value: &str) -> String
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn normalize_target(value: &str) -> String {
    match value.to_ascii_lowercase().as_str() {
        "white" => "white".to_owned(),
        "black" => "black".to_owned(),
        _ => "both".to_owned(),
    }
}
```

Where it fits

The implementation lives in the focused `web::request` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `parse_query`, `url_decode`, `hex_value`, `normalize_difficulty`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 109 of 111: `web::router::route`

Source: `src/web/router.rs:13`

Responsibility

Splits path from query values and selects the correct page, game handler, download, or 404.

Signature and data contract

```
pub(super) fn route(target: &str) -> HttpResponse
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn route(target: &str) -> HttpResponse {
    let (path, query) = target.split_once('?').unwrap_or((target, ""));
    let params = parse_query(query);
    match path {
        "/" => HttpResponse::Page(render_home("")),
        "/guide" => HttpResponse::Page(render_book("introduction")),
        "/guide/download" => HttpResponse::Download(download_book_markdown()),
        "/guide/functions.pdf" => {
            HttpResponse::Pdf(include_bytes!("../../output/pdf/GTE-Function-Appendix.pdf").to_vec())
        }
        "/load" => load_game(&params),
        "/play" => render_session(&params),
        "/move" => update_move(&params),
        "/guess-elo" => submit_elo_guess(&params),
        "/guess-opening" => submit_opening_guess(&params),
        "/new" => HttpResponse::Page(render_home("Choose a mode to start another round.")),
        _ => path
            .strip_prefix("/guide/")
            .and_then(book_chapter_index)
            .map(|index| HttpResponse::Page(render_book(&book_chapters()[index].slug)))
            .unwrap_or(HttpResponse::NotFound),
    }
}
```

Where it fits

The implementation lives in the focused `web::router` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

This module has one function; its supporting types and constants live alongside it.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 110 of 111: `web::session::sessions`

Source: `src/web/session.rs:27`

Responsibility

Lazily initializes and returns the process-wide mutex-protected browser session map.

Signature and data contract

```
pub(super) fn sessions() -> &'static Mutex<HashMap<String, Session>>
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn sessions() -> &'static Mutex<HashMap<String, Session>> {
    SESSIONS.get_or_init(|| Mutex::new(HashMap::new()))
}
```

Where it fits

The implementation lives in the focused `web::session` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `max_ply`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.

Function page 111 of 111: `web::session::max_ply`

Source: `src/web/session.rs:31`

Responsibility

Applies the mode rule that opening rounds show at most eight plies while Elo can show the full game.

Signature and data contract

```
pub(super) fn max_ply(session: &Session) -> usize
```

Read the parameter types first: owned `String` values can outlive the current call; borrowed `&str` or references read existing storage; `Option<T>` represents data that may be absent; and `Result<T, E>` makes recoverable failure explicit. The return type tells the caller what it can do next. The body below shows which paths this function uses.

Implementation, line by line

This excerpt is copied from the production source. For each `if`, `match`, or early `return`, identify what input reaches that path and what state the caller observes afterward. When a value changes, follow it through the types named in the signature.

```
pub(super) fn max_ply(session: &Session) -> usize {
    let game_count = session.game.ply_count();
    if session.mode == GameMode::Opening {
        game_count.min(8)
    } else {
        game_count
    }
}
```

Where it fits

The implementation lives in the focused `web::session` module at the source path above. Read the related architecture chapter for its larger flow and the complete codebase walkthrough for the module boundaries, related helpers, exercises, and known limits.

Other functions in this module: `sessions`.

For terminal exercises that touch this behavior, follow the exercise's `reference.md` link and compare its small contract with this production implementation.